Politechnika Warszawska

WYDZIAŁ ELEKTRYCZNY



Instytut Elektrotechniki Teoretycznej i Systemów Informacyjno-Pomiarowych

Praca dyplomowa magisterska

na kierunku Informatyka Stosowana

w specjalności Cyberbezpieczeństwo

Porównanie schematów szyfrowania plików przy użyciu fizycznych kluczy bezpieczeństwa

inż. Kacper Karaś

numer albumu 319053

promotor

dr inż. Bartosz Chaber

WARSZAWA 2026

Porównanie schematów szyfrowania plików przy użyciu fizycznych kluczy bezpieczeństwa

Streszczenie

Niniejsza praca przedstawia porównawczą analizę bezpieczeństwa i wydajności dwóch systemów szyfrowania plików: tradycyjnego standardu OpenPGP oraz nowoczesnego narzędzia age. OpenPGP to dojrzały, rozwijany od dekad protokół, którego bezpieczeństwo obciążone jest historycznymi decyzjami projektowymi oraz koniecznością zachowania wstecznej kompatybilności. Z kolei age reprezentuje minimalistyczną koncepcję sztywnego doboru bezpiecznych mechanizmów, całkowicie pozbawioną podatnych opcji konfiguracyjnych. Głównym celem pracy jest empiryczna ocena, które z tych rozwiązań lepiej odpowiada współczesnym wyzwaniom stawianym narzędziom ochrony danych.

W pierwszej części pracy zbadano efektywność czasową operacji kryptograficznych. W celu wyeliminowania zmienności środowiska i odizolowania narzutu samego formatu danych, skonstruowano dedykowany kryptograficzny klucz sprzętowy oparty na mikrokontrolerze z dedykowanym, bezpiecznym układem scalonym. Architektura ta gwarantuje, że wrażliwe operacje asymetryczne i generowanie kluczy odbywają się wewnątrz odizolowanego komponentu sprzętowego. Do obsługi klucza opracowano dedykowane aplikacje klienckie dla obu badanych programów. Wykorzystując to jednolite środowisko, przeprowadzono testy wydajnościowe na dwóch niezależnych komputerach, mierząc narzut komunikacyjny z układem sprzętowym, czas szyfrowania i deszyfrowania oraz narzut na rozmiar wynikowych plików.

Drugą, kluczową część pracy stanowi praktyczna analiza ośmiu klas podatności OpenPGP, poparta autorskimi demonstracjami w środowisku testowym. Zbadano m.in.: ataki polegające na podmianie parametrów klucza z wykorzystaniem małych podgrup matematycznych, wycieki informacji poprzez wyrocznie deszyfrujące, ataki kolizyjne na przestarzałe szyfry blokowe, łamanie haseł atakiem brutalnej siły z powodu słabych funkcji rozciągania klucza, deanonimizację odbiorców przez jawne metadane w nagłówkach, wycieki treści wywołane kompresją danych oraz podatności parsera ujawnione metodą testowania różnicowego. Dla każdego wektora zweryfikowano odporność age.

Zebrane wyniki potwierdzają, że age zapewnia wyższy poziom bezpieczeństwa przy zachowaniu co najmniej równorzędnej efektywności czasowej. Brak opcji konfiguracyjnych eliminuje błędy ludzkie i celową degradację parametrów, a jednorodny dobór nowoczesnych mechanizmów (szyfrowanie uwierzytelnione, pamięciożerna funkcja ochrony hasła, pojedyncza krzywa eliptyczna) sprawia, że badane klasy podatności są w age niemożliwe do wywołania na poziomie samej architektury. W przeciwieństwie do tego OpenPGP utrwala przestarzałe i podatne konstrukcje ze względu na wymóg obsługi starych plików. Praca dostarcza tym samym argumentów za tezą, że współczesne systemy szyfrowania plików powinny traktować age jako rozwiązanie preferowane.

Słowa kluczowe: OpenPGP, age, RFC 9580, kryptografia stosowana, analiza podatności

Comparison of File Encryption Schemes Using Physical Security Keys

Abstract

This thesis presents a comparative security and performance analysis of two file encryption systems: the traditional OpenPGP standard and the modern age tool. OpenPGP is a mature protocol developed over several decades, whose security is burdened by historical design choices and the necessity of maintaining backward compatibility. Conversely, age represents a minimalist philosophy of enforcing secure, fixed parameters, entirely devoid of vulnerable configuration options. The primary objective of this work is to empirically evaluate which of these solutions better meets the challenges of modern data protection.

The first part of the thesis examines the execution time efficiency of cryptographic operations. To eliminate environmental variability and isolate the overhead imposed strictly by the file format, a dedicated hardware cryptographic key was designed using a microcontroller paired with a secure hardware element. This architecture guarantees that sensitive asymmetric operations and key derivations are executed entirely within an isolated hardware component. Dedicated client applications were developed to interface with the hardware key for both evaluated programs. Leveraging this uniform platform, performance benchmarks were conducted across two independent host machines, measuring hardware communication latency, encryption and decryption throughput, and file storage overhead.

The second, key part of the work provides a practical analysis of eight OpenPGP vulnerability classes, supported by custom demonstrations in a test environment. The investigated vectors include: key parameter substitution attacks using small mathematical subgroups, information leakage via decryption oracles, collision attacks against legacy block ciphers, brute-force password recovery due to weak key-stretching algorithms, recipient de-anonymization through plaintext metadata headers, content leakage caused by data compression, and parser vulnerabilities identified through differential testing. For each vector, the resilience of age was evaluated.

The findings confirm age provides a higher security margin while maintaining equivalent or superior operational efficiency. Eliminating configuration options prevents human error and deliberate algorithm downgrades, while the uniform adoption of modern primitives (authenticated encryption, memory-hard password hashing, and a single elliptic curve) renders the investigated vulnerability classes structurally impossible in age. In contrast, OpenPGP perpetuates legacy and fragile constructs to support historical files. This thesis therefore substantiates the claim that modern file encryption systems should adopt age as the preferred solution.

Keywords: OpenPGP, age, RFC 9580, applied cryptography, vulnerability analysis

Spis treści

1 Wstęp **9**

2 Omówienie protokołów szyfrujących **13**
2.1 Omówienie działania protokołu OpenPGP 13
2.1.1 Kontrola procesu przez użytkownika 14
2.1.2 Wsteczna komatybilność 15
2.2 Omówienie działania protokołu age 16
2.2.1 Kontrola procesu przez użytkownika 17
2.2.2 Komatybilność narzędzia 18

3 Metodyka badań **19**
3.1 Wspólna platforma sprzętowa 19
3.1.1 Warstwa fizyczna - komunikacja 20
3.1.2 Problem różnicy obługiwanych krzywych eliptycznych 21
3.2 Klient formatu OpenPGP 21
3.2.1 Generacje certyfikatu OpenPGP 21
3.2.2 Szyfrowanie 22
3.2.3 Odszyfrowywanie 22
3.3 Klient formatu age 23
3.3.1 Strofa `pico-p256` 23
3.3.2 Szyfrowanie 24
3.3.3 Odszyfrowywanie 24
3.4 Izolacja środowiska badawczego 24
3.5 Metodyka pomiarów 25

4 Porównanie programów szyfrujących na poziomie wydajności i bezpieczeństwa **27**
4.1 Porównanie wydajnościowe 27
4.1.1 Narzut czasowy wykonywania operacji kryptograficznych z wykorzystaniem klucza sprzętowego 27
4.1.2 Porównanie czasu potrzebnego na szyfrowanie w zależności od wielkości pliku 29
4.1.3 Porównanie czasu potrzebnego na odszyfrowanie w zależności od wielkości pliku 30
4.1.4 Porównanie wielkości zaszyfrowanych plików 31

4.2 Analiza bezpieczeństwa programów szyfrujących 32
4.2.1 Omówienie literatury 33
4.2.2 Podatność: EFail / CFB Malleability 34
4.2.3 Podatność: Adaptive Chosen-Ciphertext Attack 35
4.2.4 Podatność: SWEET32 - atak urodzinowy na 3DES 35
4.2.5 Podatność: SHA-1 S2K - brute-force offline hasła 37
4.2.6 Podatność: Metadata Leak - ujawnienie odbiorcy przez Key ID 37
4.2.7 Podatność: Compression Oracle 39
4.2.8 Podatność: Parser Attack Surface 40
4.2.9 Podatność na błędną konfigurację 41
4.2.10 Ataki niezreplikowane 42

5 Podsumowanie **45**

Bibliografia **47**

Spis rysunków **51**

Spis tabel **53**

Rozdział 1

Wstęp

Z każdym rokiem ilość ataków cybernetycznych związanych z naruszeniem danych rośnie [15, 41], a metody wykorzystywane do ich przeprowadzania zmieniają się wraz z rozwojem technologii. W ostatnim roku widoczny jest trend wzrostu liczby ataków rozpoczynających się od eksploatacji aplikacji publicznie dostępnych. Liczba takich ataków wzrosła w 2025 roku o 44%, a ponad połowa wykorzystanych podatności nie wymagała uwierzytelnienia [15]. W latach 2023–2026 eksploatacja podatności odnotowała znaczny wzrost popularności jako metoda uzyskania pierwszego dostępu, wypierając nadużywanie danych uwierzytelniających, które w 2026 roku stanowiło jedynie 13% [41]. Trend ten pokazuje zmianę w podejściu atakujących od bardziej oportunistycznego wykorzystywania błędów użytkowników, związanych ze słabymi danymi logowania czy użyciem wykradzionych danych, do podejścia bardziej aktywnego, polegającego na realnym wykorzystywaniu luk w oprogramowaniu.

Zawężając obraz z globalnego na europejski zgodnie z raportem ENISA Threat Landscape 2025, Polska znajduje się w gronie pięciu państw członkowskich UE najczęściej atakowanych w sektorze administracji publicznej (15,1%) [9]. Dominującym rodzajem ataku był ransomware, stanowiąc 83,5% przypadków wdrożenia złośliwego kodu [9].

Nie można też pominąć wpływu AI a w szczególności generatywnego AI na naruszenia bezpieczeństwa. Wpływ ten koncentruje się głównie na zwiększeniu skali i szybkości operacyjnej ataków, a także w obszarze ryzyka wewnętrznego [41]. Choć AI nie zrewolucjonizowało jeszcze całkowicie natury zagrożeń, znacząco ułatwia ono działania przestępcom na różnych etapach cyklu ataku. AI było zaangażowane w co szóstym naruszeniu bezpieczeństwa danych. Obejmuje to: ataki na autoryzowane systemy AI, incydenty wynikające z tzw. Shadow AI oraz ataki, w których napastnicy wykorzystują AI do przełamywania zabezpieczeń [14]. AI jest jednak wykorzystywana nie tylko przez atakujących, ale również przez twórców aplikacji internetowych. Zjawisko vibecodingu, czyli modelu programowania, w którym deweloperzy opisują pożądaną funkcjonalność w języku naturalnym i akceptują wygenerowany przez AI kod z minimalną kontrolą, doprowadziło do znaczącego pogorszenia bezpieczeństwa wypuszczanych produktów. Problem ten jest mierzalny, kod generowany przez AI wprowadza luki bezpieczeństwa w przypadku 45% zadań, przed jakimi zostało postawione [40]. Ponadto badania wykazują, że prośby o scalenie współtworzone przez AI zawierają prawie 3 razy

więcej problemów z bezpieczeństwem niż te napisane wyłącznie przez ludzi [6], analiza 1400 aplikacji produkcyjnych zbudowanych przy użyciu platform do vibecodingu wykazała, że 65% z nich miało problemy z bezpieczeństwem, a 58% zawierało co najmniej jedną krytyczną podatność [5, 6]. Jednym z najgroźniejszych skutków vibecodingu jest fałszywe poczucie bezpieczeństwa, aż 80% spośród badanych deweloperów wierzy, że AI generuje bezpieczniejszy kod niż ludzie, co stoi w sprzeczności z danymi empirycznymi [6].

Jak widać znaczenie bezpieczeństwa cyfrowego w dzisiejszych czasach jest bardzo wysokie i wynika z wielu czynników. Jednym z problemów, na które warto spojrzeć bliżej, jest zyskujący znaczną popularność ransomware oparty na eksfiltracji danych. Ransomware jest już jednym z bardziej popularnych ataków w naszym regionie [9], a jego ewolucja w kierunku eksfiltracji danych sprawiła, że z problemu czysto technicznego stał się poważnym zagrożeniem, w którym kradzież informacji staje się ważniejszym środkiem nacisku niż samo szyfrowanie plików. Model ten polega na eksfiltracji wrażliwych danych przed uruchomieniem procesu szyfrowania, przeciwdziała to tradycyjnej metodzie obrony polegającej na przywracaniu systemów z backupu, ponieważ organizacja nadal stoi przed ryzykiem upublicznienia skradzionych danych [31]. Popularność tej metody potwierdza spadek liczby ataków, w których przeprowadzono proces szyfrowania, w 2025 odsetek ten wyniósł 50% wobec 70% w 2024 roku [30]. Dodatkowo ataki polegające jedynie na ekstrakcji danych stanowiły 6% wszystkich icydentów w 2025, co oznacza dwukrotny wzrost względem roku poprzedniego [30]. Niepokojącym jest fakt, że napastnicy działają coraz szybciej, co skraca czas dostępny na obronę, szybkość eksfiltracji danych w najszybszych atakach wzrosła czterokrotnie w ciągu 2026 roku [36]. Najszybsze 25% ataków osiąga etap kradzieży danych w zaledwie 72 minuty od momentu uzyskania dostępu (wcześniej trwało to średnio 285 minut), a 22% wszystkich incydentów kończy się eksfiltracją w czasie krótszym niż jedna godzina [36].

Przedstawiona powyżej analiza aktualnej sytuacji w sferze cyberataków uwidacznia potrzebę lepszej ochrony danych. Jednym ze sposoóbw zapewnienia lepszego bezpieczeństwa przechowywanych plików jest ich szyfrowanie metodą asymetryczną. Jednym z szeroko stosowanych rozwiązań jest używanie programu PGP (Pretty Good Privacy). Powstał on w 1991 roku, ale jego pierwszy udokumentowany standard zapisano w 1996 roku w RFC 1991 [43]; aktualnie obowiązującym standardem jest natomiast RFC 9580, wydany w lipcu 2024 roku [42]. Długa historia i bardzo szerokie użycie PGP nie wpłynęły pozytywnie na opinie dotyczące bezpieczeństwa programu, PGP było krytykowane za utrzymywanie wstecznej kompatybilności wielu niebezpiecznych mechanizmów oraz samą konstrukcją protokołu:

1. brak uwierzytelniania na poziomie wiadomości
2. problematyczna kompresji, brak ochrony nagłówków i metadanych
3. brak utajniania z przekazywaniem (Forward Secrecy)

Jego naprawa wymagałaby zerwania z dotychczasowymi standardami e-mail i MIME, co utrudnia skuteczną modernizację [12]. Dodatkową wadą jest wysoki próg wejścia przy korzystaniu z programu

na własny użytek, w badaniu opisanym w artykule "Why Johnny Still, Still Can't Encrypt: Evaluating the Usability of a Modern PGP Client" [28] tylko jednej z dziesięciu par uczestników udało się poprawnie wymienić wiadomości, głównie dzięki temu, że jedna z osób miała wcześniejszą wiedzę na temat kryptografii.

Przez długi czas, mimo wad PGP, nie istniał inny program, który mógłby skutecznie zastąpić go w roli narzędzia do szyfrowania. W 2019 sytuacja się zmieniła, kiedy Filippo Valsorda stworzył projekt age (pierwsza stabilna wersja pojawiła się w 2021), który w swoich założeniach jest wyspecjalizowany, prosty w obsłudze, stosuje najnowsze standardy bezpieczeństwa i jest skupiony na poufności [37].

Biorąc pod uwagę, że atakujący coraz częściej wykorzystują eksploatację podatności jako metodę uzyskania pierwszego dostępu, można zadać pytanie, czy używanie PGP jest bezpieczne w 2026 roku. PGP w najnowszym standardzie RFC 9580 próbuje wyeliminować od dawna znane luki bezpieczeństwa, pozostaje jednak pytanie, czy jest to wystarczające, aby zagwarantować bezpieczeństwo użytkownikom.

Celem niniejszej pracy jest porównanie obu rozwiązań, PGP oraz age, zarówno pod kątem ich wydajności, jak i przede wszystkim odporności na ataki. Przeanalizowane zostaną wektory ataków na zaszyfrowane pliki w różnych scenariuszach oraz to, jak PGP i age radzą sobie z tymi wektorami.

Rozdział 2

Omówienie protokołów szyfrujących

Przed przystąpieniem do omówienia analizy przeprowadzonych badań należy zaznajomić się z programami, które zostaną porównane. W niniejszym rozdziale opisano schemat szyfrowania plików za pomocą narzędzi PGP oraz age, z naciskiem na dokładne omówienie kroków podejmowanych podczas tego procesu oraz narzędzi kryptograficznych wykorzystywanych do uzyskania zaszyfrowanego pliku.

2.1 Omówienie działania protokołu OpenPGP

W ramach niniejszej pracy jako klient PGP wykorzystano rozwiązanie GnuPG w wersji 2.4.9, najpopularniejszą implementację standardu OpenPGP. PGP można zdefiniować jako hybrydowy systemem kryptograficzny, który łączy zalety kryptografii klucza publicznego (asymetrycznej) oraz szyfrowania symetrycznego, aby zapewnić poufność i integralność danych [42, §2]. Proces szyfrowania rozpoczyna się od przygotowania pliku źródłowego i kompresji danych. Każdy plik lub wiadomość przesyłana przez OpenPGP jest najpierw opakowywana w tzw. Literal Data Packet [42, §5.9], pakiet ten zawiera samą treść (body) oraz metadane ułatwiające jej interpretację przez odbiorcę. Metadane takie jak nazwa pliku czy data nie są chronione przez podpis cyfrowy wewnątrz tego pakietu. Po przygotowaniu danych dosłownych i nałożeniu na nie podpisu cyfrowego, strumień pakietów może zostać poddany kompresji [42, §2.1]. Wynik kompresji jest umieszczany w Compressed Data Packet [42, §5.6], pakiet ten składa się z 1-bajtowego identyfikatora algorytmu oraz skompresowanego strumienia danych. Warto zaznaczyć, że etap kompresji nie jest obowiązkowy, jednak kompresja jest domyślnie włączona.

Po przygotowaniu danych następuje krok generowania klucza sesji. Dla każdego procesu szyfrowania oprogramowanie OpenPGP generuje unikalny, losowy klucz sesyjny przy użyciu bezpiecznego generatora liczb pseudolosowych [42, §2.1, §13.10]. Klucz sesyjny musi zostać bezpiecznie przekazany odbiorcy, OpenPGP przewiduje dwa sposoby przeprowadzenia tej operacji: klucz sesyjny jest szyfrowany kluczem publicznym każdego z odbiorców [42, §5.1], dzięki temu tylko posiadacz odpowiedniego klucza prywatnego może go odczytać. Lub drugi sposób klucz sesyjny jest szyfrowany

kluczem pochodzącym z hasła podanego przez użytkownika, które przekształcane jest w klucz symetryczny za pomocą *specyfikatora S2K* (String-to-Key) [42, §3.7, §5.3].

Po ustaleniu sposobu zabezpieczenia klucza sesji jest on wykorzystywany do właściwego szyfrowania danych. W OpenPGP stosuje się specjalny wariant trybu CFB (Cipher Feedback) [42, §5.13], wykorzystujący pakiet w dwóch wersjach: v1 wykorzystującą szyfrowanie CFB oraz kod detekcji modyfikacji MDC (Modification Detection Code) [42, §5.13.1], oraz v2 wykorzystującą tryby AEAD (Authenticated Encryption with Associated Data) [42, §5.13.2].

Dokument wyjściowy takiego procesu składa się z sekwencji pakietów. Zawiera przede wszystkim jeden lub więcej pakietów przechowujących informacje o kluczu sesji, po jednym dla każdego odbiorcy lub hasła, oraz główny pakiet danych zaszyfrowanych, który po odszyfrowaniu zawiera dane skompresowane lub bezpośrednio pakiet danych dosłownych.

2.1.1 Kontrola procesu przez użytkownika

GnuPG (GPG) udostępnia użytkownikowi niezwykle szeroki wachlarz opcji i flag, które pozwalają na pełną kontrolę nad każdym aspektem kryptograficznym i technicznym procesu szyfrowania. Poniżej zaprezentowano najważniejsze kategorie tych opcji.

1. wybór algorytmów i prymitywów
   Użytkownik może zdecydować, jakie algorytmy zostaną użyte do szyfrowania symetrycznego, wybierając z listy obejmującej m.in. AES (128, 192, 256), Twofish, Camellię, a także starsze, obecnie odradzane algorytmy, takie jak TripleDES czy CAST5 [42, §9.3]. Ten sam wybór dotyczy kluczy asymetrycznych - GPG umożliwia generowanie i używanie kluczy opartych na krzywych eliptycznych (np. X25519, Ed25519, krzywe NIST i Brainpool) lub klasycznego RSA (o długości od 2048 do 4096, a nawet 8192 bitów) [42, §9.1]. Można również wymusić użycie konkretnej funkcji skrótu, np. SHA2-256, SHA2-512 czy SHA3, ale także starszych, takich jak MD5 czy SHA-1 [42, §9.5].

2. tryby szyfrowania i ochrona integralności
   Standard OpenPGP pozwala na wybór trybu pracy szyfru: dla AEAD (Authenticated Encryption with Associated Data) dostępne są tryby, takie jak OCB (domyślny dla RFC 9580), EAX czy GCM, które zapewniają jednoczesne szyfrowanie i silną ochronę integralności [42, §9.6]. Dostępny jest oczywiście również klasyczny tryb OpenPGP wykorzystujący CFB (Cipher Feedback) oraz MDC (Modification Detection Code) [42, §5.13.1].

3. kompresja i formatowanie danych
   Użytkownik może wybrać między brakiem kompresji (none) a algorytmami takimi jak ZIP, ZLIB lub BZip2 [42, §9.4, §12.3.1.1]. Dodatkowo flaga `-z` pozwala określić poziom kompresji (od 0 do 9), umożliwiając balansowanie rozmiarem pliku [34]. Natomiast flaga `--armor` pozwala na konwersję binarnego pliku zaszyfrowanego do bezpiecznej postaci tekstowej (Base64), co jest przydatne przy przesyłaniu danych kanałami tekstowymi.

4. maskowanie danych
   OpenPGP oferuje flagi zwiększające anonimowość użytkownika, takie jak `--hidden-recipient` lub `--throw-keyids`, które usuwają z zaszyfrowanego pliku identyfikatory kluczy publicznych. Dzięki temu postronny obserwator nie wie, dla kogo plik został zaszyfrowany.

5. parametry S2K
   W przypadku szyfrowania symetrycznego użytkownik może wybrać między trybami Solone, Iterowane i Solone oraz Argon2 [42, §3.7.1]. Dostępna jest również flaga `--s2k-count`, pozwalająca określić liczbę iteracji „mieszania" hasła.

Podsumowując, OpenPGP oferuje wiele kombinacji ustawień od wyboru konkretnej krzywej eliptycznej, przez tryb uwierzytelniania danych (AEAD), aż po precyzyjne maskowanie długości plików i tożsamości odbiorców. Jak widać nawet na podstawie zamieszczonych powyżej przykładów konfiguracji, GnuPG, ze swoją mnogością oferowanych opcji i możliwości dostosowania, umożliwia atakującym łatwe wprowadzenie użytkownika w błąd i zmniejszenie bezpieczeństwa szyfrowanych danych poprzez ustawienie odpowiedniej flagi.

2.1.2 Wsteczna komatybilność

Standard OpenPGP, zwłaszcza w jego najnowszej specyfikacji (RFC 9580), wspiera szereg przestarzałych algorytmów i opcji typu legacy, głównie w celu zapewnienia kompatybilności wstecznej z archiwalnymi danymi. Nowoczesne implementacje powinny unikać ich generowania, traktując je jako niebezpieczne lub niewystarczająco silne, jest to jednak jedynie sugestia, a nie silne wymuszenie. OpenPGP zapewnia wsteczną kompatybilność na wielu poziomach; poniżej przytoczono te uznane za najważniejsze.

Jak wspomniano w sekcji 2.1.1, GnuPG umożliwia wybór przestarzałych algorytmów asymetrycznych, takich jak DSA, ElGamal, RSA o długości klucza mniejszej niż 3072 bity czy Ed25519Legacy i Curve25519Legacy (wyłącznie dla kluczy w wersji 4) [42, §9.2]. GPG daje również możliwość użycia szeregu słabych algorytmów symetrycznych: IDEA, 3DES czy CAST5. Ich użycie do szyfrowania jest zabronione (MUST NOT), choć implementacje mogą je wspierać na potrzeby deszyfrowania starych wiadomości [42, §9.3, Appendix B]. Przestarzałe funkcje skrótu, MD5, SHA-1, RIPEMD-160, również pozostają dostępne [42, §9.5]. OpenPGP wspiera ponadto stare formaty plików oraz przestarzałe mechanizmy, takie jak format SED (Symmetrically Encrypted Data Packet - Tag 9), dawny format kontenera szyfrowanego, który nie zapewnia ochrony integralności. Jest on uznany za przestarzały, jednak zostanie obsłużony, jeśli użytkownik wymusi jego akceptację [42, §5.7]. Nadal umożliwione jest również wsparcie dla najprostszych trybów przekształcania hasła (S2K) [§3.7.1.1] oraz metod ochrony klucza prywatnego opartych na prostych sumach kontrolnych (MalleableCFB) [42, §3.7.2.1].

Warto zauważyć, że GnuPG oferuje flagi takie jak `--openpgp` lub `--rfc4880`, które resetują ustawienia programu do ścisłej zgodności ze standardem, co automatycznie wyłącza wiele z tych niebezpiecznych obejść [34]. Opcje te trzeba jednak znać, a użytkowników niezaznajomionych

z licznymi flagami GPG lub nieznających się na kryptografii można łatwo zmanipulować do obniżenia bezpieczeństwa szyfrowanych dokumentów. Użytkownicy mogą też nieumyślnie ustawić niebezpieczne flagi samodzielnie, wystawiając się na atak w przekonaniu, że są bezpieczni.

2.2 Omówienie działania protokołu age

W ramach niniejszej pracy wykorzystano klient age w wersji 1.3.1. age (Actually Good Encryption) to nowoczesny format i narzędzie do szyfrowania plików, zaprojektowane z myślą o prostocie, bezpieczeństwie i łatwości użycia w potokach UNIX [38]. Przejawia się to domyślnym operowaniem na standardowym wejściu i wyjściu oraz reprezentacją kluczy jako pojedynczych łańcuchów tekstowych, które można przekazywać bezpośrednio jako argumenty wiersza poleceń. Umożliwia to bezpośrednie łączenie age z innymi narzędziami operatorem potoku, bez plików pośrednich. W poniższym podrozdziale omówiono, w jaki sposób age podchodzi do szyfrowania danych oraz w jakim formacie zapisywane są zaszyfrowane pliki.

Każdy plik zaszyfrowany w formacie age składa się z dwóch głównych części: tekstowego nagłówka oraz binarnego ładunku [39]. Aby uzyskać taki plik, na początku należy wygenerować klucz pliku oraz klucz ładunku. Proces generowania *Klucza Pliku* rozpoczyna się od wytworzenia unikalnego, 128-bitowego klucza symetrycznego za pomocą generatora liczb pseudolosowych, klucz ten nigdy nie jest używany ponownie do innych plików. Na podstawie klucza pliku oraz unikalnej, 16-bajtowej wartości nonce (generowanej osobno dla każdego pliku) tworzony jest *Klucz Ładunku* przy użyciu funkcji HKDF-SHA-256 [39].

Następnie ładunek szyfrowany jest za pomocą wariantu szyfru ChaCha20-Poly1305 o nazwie STREAM [39]. Dane pliku dzielone są na fragmenty o rozmiarze 64 KiB, a każdy fragment szyfrowany jest osobno kluczem ładunku z unikalną wartością nonce. Nonce składa się z 12 bajtów: 11-bajtowego licznika bloków (big-endian) oraz 1 bajtu flagi Dzięki tagom Poly1305 umieszczanym na końcu każdego fragmentu age wykrywa próby zmiany kolejności bloków, ich usunięcia lub ucięcia pliku. Deszyfrowanie kończy się błędem, jeśli program nie napotka poprawnie oznaczonego ostatniego fragmentu.

Przejdźmy do tego, w jaki sposób skonstruowany jest tak zaszyfrowany plik. Zacznijmy od nagłówka, który zawiera instrukcje określające, jak „odpakować" klucz pliku dla poszczególnych odbiorców [39]. Nagłówek zawsze rozpoczyna się od ciągu `age-encryption.org/v1`. Każdy odbiorca dokumentu ma własną strofę (stanza), rozpoczynającą się od symbolu `->`. Strofa zawiera argumenty określające typ odbiorcy oraz zaszyfrowany klucz pliku zakodowany w formacie Base64. Nagłówek kończy się kodem uwierzytelniającym HMAC-SHA-256, obliczonym nad całym nagłówkiem. Klucz do HMAC wyprowadzany jest z klucza pliku, co gwarantuje, że lista odbiorców nie została zmanipulowana.

Standard age definiuje kilka natywnych sposobów zabezpieczenia klucza pliku. Pierwszym jest klasyczne szyfrowanie asymetryczne oparte na krzywej Curve25519 [39], dla każdej strofy generowany jest efemeryczny klucz prywatny, a klucz pliku szyfrowany jest algorytmem ChaCha20-Poly1305

z wykorzystaniem wspólnego sekretu (Diffie-Hellman). Jeżeli natomiast plik zabezpieczany jest hasłem, age stosuje unikalną sól (16 bajtów) oraz funkcję scrypt w celu utrudnienia ataków siłowych (brute-force). Warto zauważyć, że w nagłówku może znajdować się tylko jedna strofa typu scrypt. age przygotowany jest również na erę komputerów kwantowych, umożliwiając obsługę hybrydowego szyfrowania post-kwantowego ML-KEM-768 z X25519 (X-Wing).

Najciekawszy z punktu widzenia niniejszej pracy jest natomiast tryb Tagged Recipients - specjalny typ odbiorców zaprojektowany z myślą o współpracy z kluczami sprzętowymi (np. YubiKey), gdzie proces deszyfrowania może wymagać fizycznej obecności użytkownika (np. dotknięcia urządzenia). Główną różnicą między Tagged Recipients a standardowymi typami (jak X25519) jest możliwość zidentyfikowania odbiorcy przed próbą deszyfrowania. W standardowym typie X25519 strofy są anonimowe, program musi próbować odszyfrować każdą z nich swoim kluczem prywatnym. Tagged Recipients oparty jest na HPKE i wykorzystuje P-256 ECDH w celu zapewnienia kompatybilności z istniejącym sprzętem, opcjonalnie hybrydyzowany z ML-KEM-768 dla zapewnienia odporności kwantowej. Tagged Recipients są ściśle powiązane z architekturą wtyczek w age. Zakodowanie odbiorcy może zostać zinterpretowane przez konkretną wtyczkę (jak `age-plugin-yubikey`), co pozwala na obsługę specyficznych urządzeń bez konieczności wprowadzania zmian w samym rdzeniu programu age. Będzie to istotne w dalszej części pracy.

2.2.1 Kontrola procesu przez użytkownika

Narzędzie age zostało zaprojektowane z myślą o maksymalnej prostocie i bezpieczeństwie, co przejawia się w całkowitym braku opcji modyfikacji konfiguracyjnych przez użytkownika [38, 39]. Aby pokazać to minimalistyczne podejście do funkcji oddanych użytkownikowi, poniżej omówiono wszystkie flagi konfiguracyjne age.

Zacznijmy od opcji pozwalających zarządzać odbiorcami i tożsamością: `--recipient` pozwala na dodanie jednego lub wielu odbiorców (kluczy publicznych), `--recipients-file` umożliwia wczytanie listy odbiorców z pliku, w którym każdy klucz publiczny znajduje się w osobnym wierszu, natomiast `--identity` służy do wskazania pliku tożsamości (klucza prywatnego) podczas deszyfrowania.

Aby wymusić szyfrowanie hasłem zamiast kluczem publicznym, należy użyć flagi `--passphrase`.

Na koniec pozostają dwie flagi związane z formatowaniem wyjścia: `--armor` powoduje wygenerowanie zaszyfrowanego pliku w tekstowym formacie PEM zamiast binarnego, a `--output` służy do określenia nazwy pliku wyjściowego.

Jak widać, liczba opcji sprowadzona jest do minimum, co daje nawet najmniej zaznajomionemu z kryptografią użytkownikowi jasny obraz tego, za co odpowiada dana flaga. Co najważniejsze, za pomocą flag nie można wymusić słabszej jakości kryptograficznej.

2.2.2 Komatybilność narzędzia

Przy tak młodym rozwiązaniu jak age trudno mówić o kompatybilności wstecznej. Kwestią, którą można poruszyć jest to, jak age radzi sobie z problemem obsługi każdego nowego urządzenia czy wprowadzenia nowego algorytmu. Zamiast obciążać główny kod programu, age wykorzystuje architekturę wtyczek. Przykładem jest obsługa kluczy post-kwantowych: choć natywne wsparcie dodano w wersji 1.3.0, starsze wersje age mogą obsługiwać te klucze po doinstalowaniu odpowiedniej wtyczki, o ile wspierają sam mechanizm wtyczek [38].

Dodatkowo age zaprojektowano tak, aby współpracował z powszechnie używanymi standardami tożsamości, co ułatwia jego adopcję - przykładem mogą być klucze SSH. Narzędzie natywnie wspiera szyfrowanie dla publicznych kluczy SSH (ssh-rsa oraz ssh-ed25519), co pozwala na wykorzystanie istniejącej infrastruktury kluczy bez konieczności ich konwersji.

Podsumowując, age przedkłada prostotę i bezpieczeństwo nad ekstremalną kompatybilność wsteczną, a ewentualne potrzeby wsparcia specyficznych rozwiązań (jak klucze sprzętowe czy nowe algorytmy) realizuje poprzez elastyczny system wtyczek.

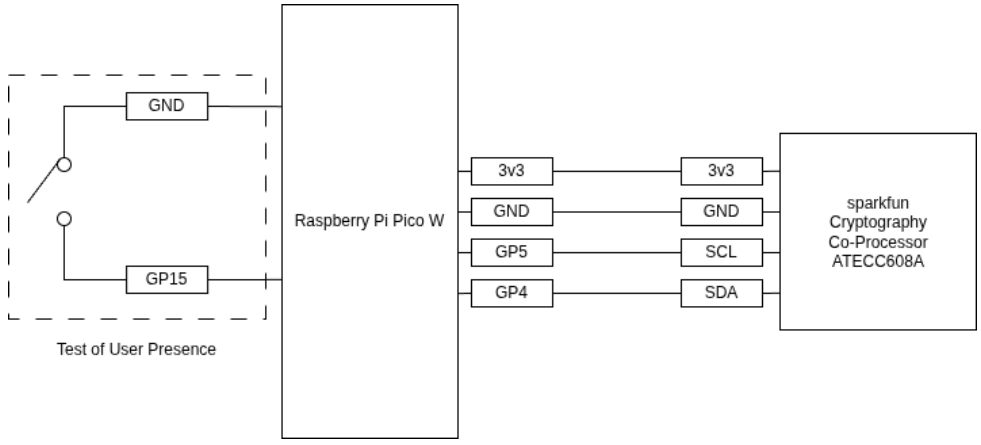
Rozdział 3

Metodyka badań

Aby podejść do porównania obu programów, należało zaprojektować kontrolowane środowisko pomiarowe, którego zadaniem jest umożliwienie rzetelnego porównania efektów działania obu narzędzi na szyfrowanym pliku. Konstrukcja tego stanowiska została podporządkowana zasadzie izolacji badanej zmiennej: wszystkie elementy niezwiązane z formatem pliku i doborem prymitywów symetrycznych zostały ujednolicone, tak aby zmierzone różnice można było przypisać wyłącznie różnicom między age a OpenPGP, a nie zmienności środowiska. Dlatego na potrzeby przeprowadzonego badania skonstruowano sprzętowy klucz kryptograficzny oparty na Raspberry Pi Pico W z układem ATECC608A oraz odpowiednie aplikacje klienckie dla age i OpenPGP, umożliwiające wykonanie za ich pomocą stosownych pomiarów. Niniejszy rozdział zawiera opis wykorzystanego środowiska badawczego, od wspólnej warstwy sprzętowej po poszczególne aplikacje klienckie.

3.1 Wspólna platforma sprzętowa

Warstwę stałą stanowi klucz sprzętowy: firmware napisany w języku C, uruchamiany na mikrokontrolerze RP2040, komunikujący się z układem ATECC608A poprzez magistralę I2C i udostępniający



Rysunek 1. Schemat sprzętowego klucza kryptograficznego

operacje kryptograficzne hostowi za pomocą protokołu ramek bajtowych przesyłanych po USB CDC. Schemat urządzenia jest przedstawiony na Rys. 1. Urządzenie nie szyfruje plików samodzielnie, realizuje jedynie operację kryptograficzną ECDH (Elliptic Curve Diffie-Hellman) z użyciem klucza prywatnego, który jest bezpiecznie przechowywany wewnątrz układu ATECC608A. Oznacza to, że układ otrzymuje przez USB wyłącznie 64-bajtowy klucz publiczny drugiej strony i zwraca 32-bajtowy sekret współdzielony, nie mając przy tym dostępu do treści pliku ani nie wykonując pozostałych etapów procesu, takich jak: wyprowadzenie klucza szyfrującego, sam proces szyfrowania lub deszyfrowania danych oraz budowa formatu pliku wynikowego. Wszystkie te operacje są wykonywane programowo na komputerze hosta. Rolą układu ATECC608A jest wyłącznie jednokrotne wykonanie mnożenia punktu na krzywej eliptycznej kluczem prywatnym, który w żadnym momencie nie opuszcza jego pamięci. Rozwiązanie przewiduje możliwość przechowywania do ośmiu par kluczy na chipie.

W celu zachowania standardów komercyjnych kluczy kryptograficznych do mikrokontrolera podłączono przycisk (widoczny na Rys. 1), którego naciśnięcie służy za potwierdzenie, że użytkownik faktycznie znajduje się fizycznie przy urządzeniu podczas próby wykonania polecenia związanego z użyciem funkcji kryptograficznych. Chroni to przed atakami, w których złośliwe oprogramowanie na urządzeniu użytkownika mogłoby automatycznie użyć klucza bez jego wiedzy.

Warto zauważyć, że podczas wykonywania pomiarów test obecności użytkownika został wyłączony, aby narzut czasowy wynikający z tej interakcji nie wpłynął na wyniki badań.

3.1.1 Warstwa fizyczna - komunikacja

Podczas korzystania z urządzenia występują dwa węzły komunikacji: między Raspberry Pi Pico a urządzeniem użytkownika oraz między Raspberry Pi Pico a układem ATECC608A.

Raspberry Pi Pico W, podłączone do komputera przez USB, rejestruje się jako urządzenie USB CDC (Communications Device Class). Cała konfiguracja mikrokontrolera jako urządzenia USB została wykonana z wykorzystaniem narzędzia TinyUSB [33]. Firmware obsługuje komunikację USB w pętli głównej poprzez wywołanie funkcji `tud_task()`, a odbiór danych wyzwala funkcję zwrotną `tud_cdc_rx_cb()`, która gromadzi bajty w buforze aż do uformowania kompletnej ramki. Komunikacja oparta jest na odpowiednio skonstruowanych ramkach binarnych: pierwszy bajt odpowiada za kod komendy, dwa kolejne bajty określają długość przesyłanych danych, następnie zawarte są dane o wspomnianej wcześniej długości, a ostatnie dwa bajty to suma kontrolna CRC-16. Odpowiedź również konstruowana jest w postaci ramki binarnej, w której pierwszy bajt to status zleconego zadania, dwa kolejne bajty to długość odpowiedzi, następnie umieszczana jest sama odpowiedź, a ostatnie dwa bajty to ponownie suma kontrolna CRC-16.

Raspberry Pi Pico W komunikuje się z układem ATECC608A poprzez magistralę I2C (Inter-Integrated Circuit). Protokół komunikacji z układem na poziomie I2C wymaga ścisłego przestrzegania sekwencji stanów: obudzenia urządzenia, wysłania komendy, oczekiwania przez czas wyznaczony w dokumentacji na wykonanie komendy, odczytania odpowiedzi oraz ustawienia ATECC608A w stan

bezczynny. Obsługę operacji związanych z układem ATECC608A zrealizowano za pomocą biblioteki CryptoAuthLib firmy Microchip [22].

3.1.2 Problem różnicy obługiwanych krzywych eliptycznych

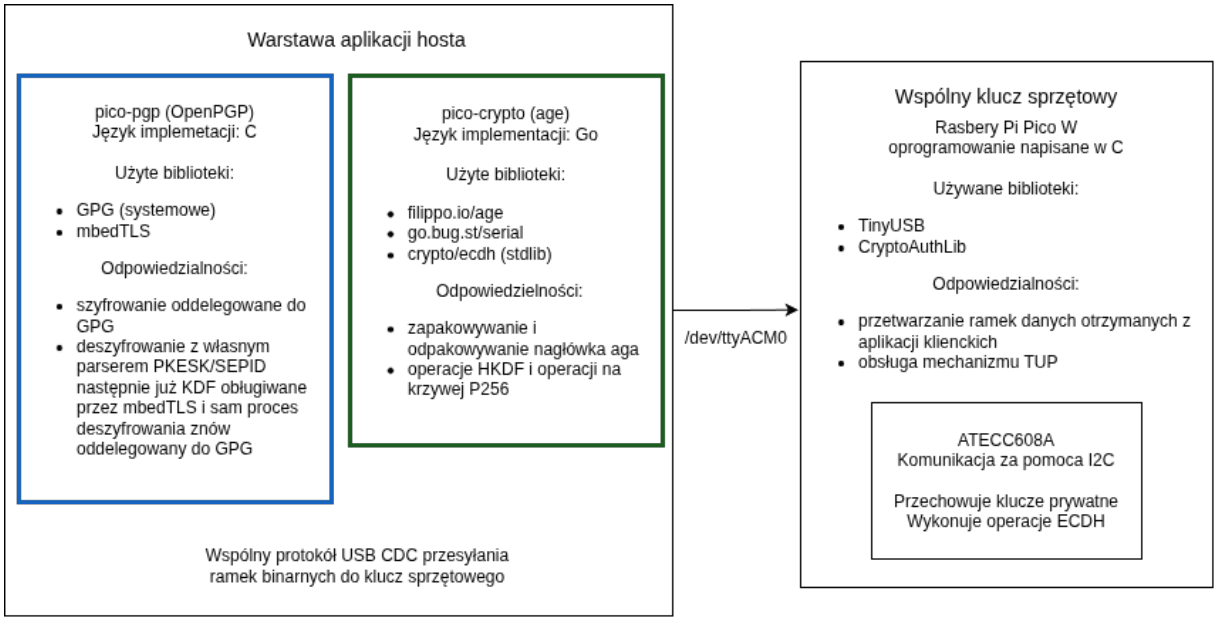
Jak wspomniano w sekcji 2.2, opisującej teorię stojącą za wykorzystaniem age do szyfrowania, klasyczne szyfrowanie asymetryczne oparte jest na krzywej Curve25519 (X25519). Układ ATECC608A nie obsługuje tej krzywej - ma dostęp jedynie do P-256 (secp256r1) [21]. age oczekuje 32--bajtowego klucza publicznego w formacie Montgomery, natomiast ATECC608A generuje klucze 64-bajtowe (X||Y, postać Weierstrassa), przez co nie jest możliwe użycie klucza P-256 jako klucza X25519 — biblioteka age odrzuciłaby taki klucz na etapie walidacji. Obsługa Curve25519 w elementach bezpiecznych nie jest powszechna, większość tanich, szeroko stosowanych układów realizuje jedynie krzywe SECP, choć istnieją wyjątki, takie jak NXP EdgeLock SE050 [25], obsługujący sprzętowo Curve25519 i Ed25519. Wybór ATECC608A był świadomym kompromisem kosztu chipsetu, jego dostępnością oraz dojrzałością obsługiwanej biblioteki. Aby obejść problem niezgodności matematycznej, zdefiniowano własną strofę `pico-p256`; dokładny opis tego zagadnienia znajduje się w sekcji 3.3.1 poniżej.

3.2 Klient formatu OpenPGP

Klient OpenPGP zaimplementowano w języku C, przyjmując architekturę minimalizującą ryzyko: szyfrowanie powierzono systemowemu GPG, natomiast deszyfrowanie realizuje własny program w C, wspomagany sprzętowym ECDH. Rys. 2 przedstawia jak, klient dla PGP wkoponowuje się w całą architekturę środowiska badawczego. Podział ten uzasadniony jest celem badania, mierzona jest efektywność formatu, a nie poprawność autorskiej implementacji szyfrowania, którą zapewnia dojrzała, referencyjna implementacja GnuPG.

3.2.1 Generacje certyfikatu OpenPGP

Aby GPG mógł szyfrować kluczem przechowywanym w chipsecie ATECC608A, jednorazowo budowany jest certyfikat OpenPGP zawierający klucz publiczny urządzenia. Program konstruuje pakiet Public-Key (tag 6, wersja 4, algorytm 18 - ECDH, klucz P-256 zakodowany jako MPI [42]) oraz pakiet User ID, a powstały certyfikat jest importowany do pęku kluczy GnuPG. W tym miejscu ujawnia się istotne ograniczenie sprzętowe: ATECC608A nie udostępnia operacji służącej do bezpiecznego generowania podpisu ECDSA (Elliptic Curve Digital Signature Algorithm) dla wiadomości przy użyciu klucza prywatnego przechowywanego sprzętowo, co jest konieczne aby uzyskać podpis własny OpenPGP, wobec czego certyfikat pozbawiony jest podpisu własnego. Rozwiązaniem docelowym jest model rozdzielenia kluczy, programowy klucz główny wyłącznie do podpisu certyfikatu oraz sprzętowy podklucz ECDH do szyfrowania, w którym prywatny klucz szyfrujący nadal nigdy nie opuszcza układu.



Rysunek 2. Wysokopoziomowy diagram architektury całego środowiska użytego do porównania programów

3.2.2 Szyfrowanie

Szyfrowanie plików realizowane jest w całości programowo przez narzędzie GnuPG na stacji roboczej, bez konieczności udziału klucza sprzętowego. Wynika to z faktu, że proces ten wymaga wyłącznie jawnego klucza publicznego odbiorcy (w testowanym scenariuszu: krzywa NIST P-256). W pierwszym kroku GPG przygotowuje symetryczny klucz sesyjny. W tym celu program generuje jednorazową parę kluczy efemerycznych i wyznacza wspólny sekret za pomocą protokołu ECDH. Na jego podstawie funkcja KDF wyprowadza klucz pośredni [42, §11.5], który służy do zabezpieczenia losowego klucza sesyjnego algorytmem AES-256 Key Wrap [13]. Tak przygotowany blok danych tworzy nagłówek pliku w postaci pakietu PKESK. Następnie właściwa treść pliku jest szyfrowana wygenerowanym kluczem sesyjnym przy użyciu algorytmu AES-256-CFB z kontrolą integralności MDC. W efekcie powstaje samodzielny plik `.pgp`, którego późniejsze odczytanie będzie wymagało operacji deszyfrowania opisaną w sekcji 3.2.3.

3.2.3 Odszyfrowywanie

Ścieżka deszyfrowania odtwarza pełny łańcuch kryptograficzny OpenPGP przy użyciu kodu własnego oraz biblioteki mbedTLS [20]. Program parsuje plik `.pgp`, wydobywa z pakietu PKESK [42, §5.1.5] efemeryczny klucz publiczny oraz opakowany klucz sesyjny, po czym przekazuje klucz efemeryczny do układu za pomocą wysłania ramki binarnrj z kodem komendy `CMD_ECDH_REQUEST`. Z uzyskanego sekretu współdzielonego wyprowadzany jest klucz opakowujący funkcją OpenPGP ECDH KDF [§11.5]: wejście funkcji skrótu stanowi ustalona konkatenacja bajtów (przedrostek, OID krzywej nistp256, oktety algorytmu, odcisk klucza, etykieta „Anonymous Sender", ponownie odcisk),

a klucz opakowujący jest skrótem SHA-256 tej konkatenacji połączonej z sekretem współdzielonym. Klucz sesyjny odzyskiwany jest algorytmem AES-256 Key Unwrap za pomocą audytowanej funkcji `mbedtls_nist_kw`. Następnie z pakietu SEIPD [§5.13.1] parsowane są wektor inicjujący oraz szyfrogram, ładunek odszyfrowywany jest w trybie AES-256-CFB, a jego integralność weryfikuje kontrola MDC (skrót SHA-1).

3.3 Klient formatu age

Klient age został w całości zaimplementowany w języku Go jako samodzielne narzędzie wiersza poleceń, które opakowuje referencyjną bibliotekę `filippo.io/age` [38] i rozszerza ją o obsługę klucza sprzętowego poprzez dwa interfejsy rozszerzeń przewidziane przez tę bibliotekę: `age.Recipient` (szyfrowanie) oraz `age.Identity` (deszyfrowanie). Rys. 2 przedstawia, jak klient age wkomponowuje się w całość architektury środowiska badawczego. Dzięki temu warstwa formatu pliku `.age` (nagłówek, strumieniowa konstrukcja ładunku oraz weryfikacja integralności) pozostaje w gestii audytowanej biblioteki, a kod własny obejmuje wyłącznie sposób opakowania i odzyskania klucza pliku przy użyciu układu ATECC608A. Ogranicza to powierzchnię własnej implementacji kryptograficznej do niezbędnego minimum.

3.3.1 Strofa `pico-p256`

Specyfikacja age [39] definiuje otwarte rozszerzenie własnego formatu poprzez możliwość zdefiniowania własnej strofy. Na potrzeby realizowanego projektu utworzono własną definicję strofy, składającą się z efemerycznego klucza publicznego oraz numeru slotu, w którym zapisany jest klucz wykorzystywany przez użytkownika podczas szyfrowania danych.

Warto odnieść przyjęte rozwiązanie do natywnej strofy P-256, którą specyfikacja age zdefiniowała pod nazwą `p256tag` w sierpniu 2025 roku [1]. Rozwiązanie `pico-p256` powstało niezależnie od tej strofy i nie stanowi jej adaptacji; zbieżność ogólnego wzorca, zdefiniowania własnej strofy dla klucza P-256 przechowywanego sprzętowo, wynika ze wspólnej inspiracji wcześniejszym `age-plugin-yubikey` [11], a nie z odwzorowania standardu. Oba rozwiązania różnią się przede wszystkim konstrukcją.

Strofa `p256tag` opiera się na mechanizmie HPKE, podczas gdy `pico-p256` wykorzystuje prostszą konstrukcję: z sekretu ustanowionego protokołem ECDH wyprowadzany jest funkcją HKDF-SHA-256 klucz opakowujący, którym klucz pliku jest szyfrowany algorytmem ChaCha20-Poly1305 - tym samym AEAD, którego age używa w natywnej strofie X25519. Różnica dotyczy również formatu klucza publicznego: `p256tag` oczekuje punktu krzywej w postaci skompresowanej, natomiast układ ATECC608A zwraca współrzędne w postaci nieskompresowanej, z której korzysta bezpośrednio `pico-p256`. Najistotniejsza różnica znajduje się w architekturze strofy i dotyczy sposobu wskazania klucza po stronie odbiorcy. Specyfikacja `p256tag` celowo nie definiuje sposobu kodowania tożsamości sprzętowej, pozostawiając tę kwestię warstwie wtyczki, a w samej strofie umieszcza jedynie skrócony

znacznik identyfikujący odbiorcę, wzorzec przejęty dla zgodności wstecznej z `age-plugin-yubikey`. Przyjęte w pracy rozwiązanie realizuje ten sam cel odmiennie: numer slotu układu ATECC608A, w którym przechowywany jest klucz prywatny odbiorcy, jest zapisywany wprost w strofie pliku, dzięki czemu operacja deszyfrowania nie wymaga od odbiorcy znajomości lokalizacji klucza, którym szyfrowany był plik. Jest to świadomy wybór projektowy przenoszący zarządzanie slotem z warstwy tożsamości do jawnego pola nagłówka pliku, uzasadniony minimalizacją złożoności narzędzia. Należy przy tym odnotować, że wybór ten pociąga za sobą kompromis w zakresie prywatności. Specyfikacja `p256tag` zaznacza, że obecność znacznika odbiorcy w strofie osłabia anonimowość i nieodróżnialność szyfrogramu, umożliwiając sprawdzenie, czy dana wiadomość była adresowana do konkretnego odbiorcy, jeszcze przed próbą deszyfrowania. Jawny numer slotu w strofie `pico-p256` ma tę samą właściwość, a nawet ujawnia nieco więcej informacji o strukturze klucza odbiorcy. W rozważanym modelu zastosowania, narzędzie pomiarowe oraz osobiste szyfrowanie plików, utrata anonimowości szyfrogramu nie stanowi jednak istotnego zagrożenia.

3.3.2 Szyfrowanie

Operacja szyfrowania rozpoczyna się od wygenerowania efemerycznej pary kluczy P-256 przez oprogramowanie klienckie. Oprogramowanie wykonuje operację ECDH między kluczem efemerycznym a kluczem publicznym pobranym z podłączonego klucza kryptograficznego, a z uzyskanego sekretu współdzielonego wyprowadza klucz opakowujący funkcją HKDF-SHA-256. Klucz pliku szyfrowany jest następnie algorytmem ChaCha20-Poly1305. Wynik zapisywany jest jako niestandardowa strofa typu `pico-p256`.

3.3.3 Odszyfrowywanie

Operacja deszyfrowania rozpoczyna się od odczytania przez aplikację kliencką efemerycznego klucza publicznego oraz numeru slotu ze strofy, po czym aplikacja przekazuje ten klucz do układu ATECC608A. Układ wykonuje operację ECDH z przechowywanym w nim kluczem prywatnym i zwraca sekret współdzielony - sam klucz prywatny nigdy nie opuszcza układu. Klient pobiera klucz publiczny slotu, odtwarza identyczny klucz opakowujący funkcją HKDF-SHA-256 i odzyskuje klucz pliku algorytmem ChaCha20-Poly1305, którego tag uwierzytelniający jednocześnie potwierdza integralność opakowanego klucza. Zapisanie numeru slotu bezpośrednio w strofie sprawia, że deszyfrowanie nie wymaga od użytkownika żadnej wiedzy o strukturze klucza — cała złożoność ukryta jest wewnątrz narzędzia klienckiego.

3.4 Izolacja środowiska badawczego

Kluczową konsekwencją metodologiczną powyższej architektury jest to, że oba programy (age i GnuPG) dzielą identyczną operację asymetryczną: ten sam układ, te same komendy oraz to samo opóźnienie wynikające z narzutu komunikacji z kluczem. Czas operacji ECDH jest zatem

wielkością stałą, a wszelkie zmierzone różnice w wydajności całego procesu wynikają wyłącznie z warstwy formatu: struktury pakietów, doboru KDF, szyfru symetrycznego (ChaCha20-Poly1305 wobec AES-256-CFB), narzutu nagłówka oraz złożoności parsowania. Pozwala to obiektywnie porównać oba programy przy ograniczeniu czynników zewnętrznych.

3.5 Metodyka pomiarów

Pomiary prowadzono bezpośrednio poprzez zestaw funkcji napisanych dla OpenPGP w C, a dla age w Go, a następnie uruchamiane przez skrypt bashowy. Jak wspomniano w sekcji 3.1, na potrzeby pomiarów wyłączono wymóg potwierdzenia obecności użytkownika przy przeprowadzaniu operacji kryptograficznych, ponieważ narzut interakcji z kluczem dominowałby nad rzeczywistym czasem operacji kryptograficznej dla małych plików i zniekształcałby porównanie. Czas mierzono zegarem monotonicznym (`CLOCK_MONOTONIC` [18]), raportując medianę z serii przebiegów po odrzuceniu przebiegów rozgrzewkowych, co ogranicza wpływ inicjalizacji pamięci podręcznej oraz wahań szeregowania procesów. Testy wykonano na jednolitym zbiorze rozmiarów plików (100 B, 1 KiB, 64 KiB, 1 MiB, 10 MiB, 100 MiB), mierząc osobno poszczególne etapy: czas operacji ECDH, szyfrowanie pliku oraz odszyfrowywanie.

Pomiary wykonano na dwóch maszynach, aby sprawdzić, czy różnice w możliwościach sprzętowych przekładają się na różnice w czasie mierzonych operacji. Pierwsza maszyna to komputer stacjonarny z dwunastowątkowym procesor AMD Ryzen 5 9600X, kartę graficzną NVIDIA GeForce RTX 5070 oraz 32 GB RAM DDR5 7000MHz. Druga maszyna to laptop ThinkPad T14s Gen 2i z ośmiowątkowym procesorem Intel Core i7-1165G7 11. generacji, kartę graficzną Intel Iris Xe Graphics oraz 16 GB DDR4 RAM 33MHz. Obie maszyny pracowały pod kontrolą systemu operacyjnego Linux z jądrem w wersji 7.0.9-arch2-1.

Rozdział 4

Porównanie programów szyfrujących na poziomie wydajności i bezpieczeństwa

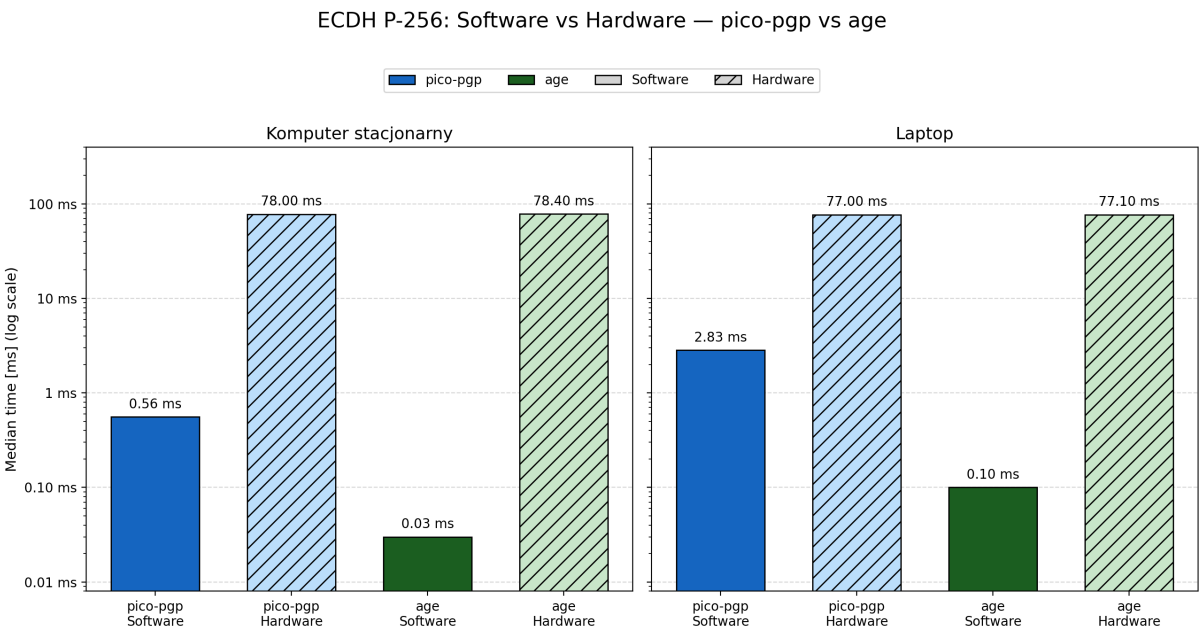
W niniejszym rozdziale omówiono porównanie OpenPGP i age na dwóch płaszczyznach: wydajności czasowej wykonywanych operacji oraz bezpieczeństwa stosowania obu programów do szyfrowania plików. Na początku przedstawiono wnioski z badań przeprowadzonych z wykorzystaniem klucza sprzętowego. W trakcie badania mierzono nakład czasowy operacji ECDH, zarówno z użyciem fizycznego klucza, jak i operacji wykorzystującej wyłącznie systemowe rozwiązania omawianych programów, a także szyfrowanie plików oraz ich odszyfrowywanie. Dodatkowo porównano rozmiar szyfrogramów. W sekcji omawiającej bezpieczeństwo programów zakwalifikowano 11 podatności dotyczących szyfrowania plików i poddano je replikacji dla OpenPGP oraz age. Zarówno w pomiarach wydajności jak i w trakcie analizy bezpieczeństwa korzystano z GPG w wersji 2.4.9 oraz age w wersji 1.3.1. Przejdźmy zatem do omówienia zebranych danych pomiarowych.

4.1 Porównanie wydajnościowe

Sekcja ta ma za zadanie zaprezentować wyniki otrzymane z pomiarów oraz opatrzyć je krótkim komentarzem. Dokładny opis stanowiska pomiarowego oraz metodyki pomiarów znajduje się w rozdziale 3. Warto jednak dodać, że dla OpenPGP wykonywano 20 pomiarów i na ich podstawie wyliczano medianę, natomiast dla age wykorzystano mechanizm benchmarków języka Go, który automatycznie wyznacza liczbę iteracji dla zadanego czasu wykonywania każdego testu - w niniejszej pracy było to 10 sekund.

4.1.1 Narzut czasowy wykonywania operacji kryptograficznych z wykorzystaniem klucza sprzętowego

Pierwszym mierzonym parametrem był narzut czasowy wymuszany przez konstrukcję wykorzystywanego klucza sprzętowego. Wybrano do tego celu operację uzgadniania wspólnego klucza sesji z wykorzystaniem protokołu ECDH. Operację tę wybrano ze względu na jej częste wykorzystanie



Rysunek 3. Porównianie czasu wykonania operacji ECDH systemowo oraz sprzętowo dla GPG i age na komputerze stacjonarnym i laptopie

podczas szyfrowania i deszyfrowania oraz na fakt, że klucz sprzętowy musi przeprowadzić komunikację z układem ATECC608A, a narzut tej komunikacji wpływa na wykonanie dowolnej operacji kryptograficznej z użyciem klucza sprzętowego. Tak zmierzony czas zestawiono z czasem wykonania operacji ECDH programowo. W tym pomiarze mierzony jest czas jednego uzgodnienia klucza ECDH na krzywej P-256, na które składają się dwa mnożenia skalarne: wyznaczenie klucza efemerycznego oraz obliczenie wspólnego sekretu, zakończone serializacją współrzędnej sekretu do 32 bajtów. Klucz publiczny odbiorcy jest w obu przypadkach przygotowany poza pętlą pomiarową, więc pomiar nie obejmuje jego parsowania ani walidacji. Klient age wykorzystuje w tych obliczeniach pakiet `crypto/ecdh` [35] ze standardowej biblioteki Go, natomiast po stronie PGP operacje realizowane są przy użyciu biblioteki `mbedTLS` [20].

Na Rys. 3 zestawiono pomiary wykonane na komputerze stacjonarnym oraz laptopie. Zaczynając od komputera stacjonarnego operacja ECDH z wykorzystaniem systemowego GPG została wykonana średnio w czasie 0,562 ms, w porównaniu z age, dla którego średnio ta operacja trwała 33,736 µs — 16,66 razy szybciej. Dla ECDH wykonanego z użyciem klucza sprzętowego pomiar dla GPG wyniósł 78,002 ms, a dla age 78,407 ms, co daje jedynie 0,405 ms różnicy między pomiarami.

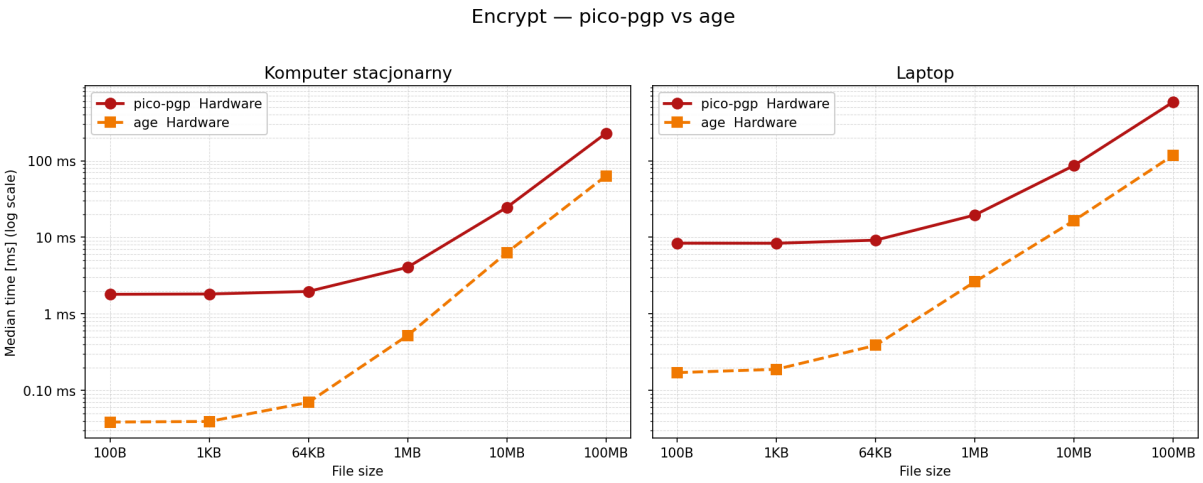
Dla porównania operacja ECDH wykonywana na laptopie z wykorzystaniem systemowego GPG wykonała się średnio w czasie 2,833 ms, w porównaniu z age, dla którego operacja ta trwała średnio 99,689 µs — 28,42 razy szybciej. Dla ECDH wykonanego z użyciem klucza sprzętowego pomiar dla GPG wyniósł średnio 77,037 ms, a dla age średnio 77,111 ms, co daje jedynie 0,074 ms różnicy między pomiarami.

Na podstawie pomiarów można stwierdzić, że istnieje stały narzut czasowy związany z korzystaniem z klucza sprzętowego, wynoszący między 77 a 78 ms. Ponieważ na dwóch różnych maszynach pomiar tej operacji różni się jedynie o około 1 ms, można było uznać, że narzut ten wynika wyłącznie z użycia fizycznego klucza, a nie z innych czynników zewnętrznych. Z tej racji dokonano pomiarów narzutu komunikacji I2C oraz USB na obu urządzeniach. Pomiar został wykonany poprzez zmierzenie czasu od momentu wysłania komunikatu do klucza systemowego do otrzymania od niego komunikatu zwrotnego, dzięki temu dostaliśmy czas przeprowadzenia całego procesu komunikacj z urządzeniem. Do klucza była wysylana specjalna ramka danych z poleceniem `CMD_GET_I2C_TIME`, które zwraca czas jaki był potrzebny na otrzymanie zwrotnej wiadomości z układem ATECC608A. Mając czas przeprowadzenia całej drogi komunikacji i czs komunikacji z ATECC608A można było wyliczyć czas przeznaczony na komunikacje USB między maszyną hostującą oprogramowanie, a kluczem systemowym. Pomiary zostały wykonane 10 razy i została wyciągnięta z nich średnia. Dla komuputera stacjonarnego komunikacja I2C wynosiła średnio 56 ms, a komunikacja USB średnio 22 ms, natomiast dla laptopa komunikacja I2C wynosiła ponownie średnio 56 ms a komunikacja po USB średnio 21 ms. W obu przypadkach około 72% całego czasu komunikacji stanowił nakład komunikacji I2C. Z racji na brak znaczących różnic miedzy klientem C oraz Go test pomiarowy został wykonany jedynie na kliencie w Go z racji na prostrzą implementacje.

4.1.2 Porównanie czasu potrzebnego na szyfrowanie w zależności od wielkości pliku

Następnym mierzonym parametrem był czas wykonania operacji szyfrowania przez GPG oraz age. Jak wspomniano w sekcji 3.5, czas mierzono dla sześciu rozmiarów wygenerowanych plików. Ponownie należy zaznaczyć, że z pomiarów całkowicie wyłączono operacje kryptograficzne, w których miałby brać udział klucz sprzętowy, tak aby wykonane pomiary odzwierciedlały jedynie czas odizolowanej operacji szyfrowania.

Na Rys. 4 przedstawiono wyniki pomiarów czasu szyfrowania wykonane na komputerze stacjonarnym oraz laptopie. Omówione zostaną na początku pomiary zebrane dla komputera stacjonarnego. Zgodnie z oczekiwaniami, wraz ze wzrostem rozmiaru szyfrowanego pliku rośnie również czas szyfrowania. Dla każdego rozmiaru age uzyskuje szybsze wyniki, od 46,59 raza szybciej dla najmniejszych plików do 3,6 raza szybciej dla największych. Jak widać, przyspieszenie maleje wraz ze wzrostem rozmiaru pliku, a jego kolejne spadki stają się coraz mniejsze. Można wnioskować, że dla odpowiednio dużych plików sama ilość danych do zaszyfrowania będzie na tyle duża, że różnice między procesami realizującymi tę operację staną się znikome, a czas szyfrowania będzie de facto identyczny. Warto dodatkowo zwrócić uwagę na minimalny wzrost przyspieszenia między szyfrowaniem plików 100 B i 1 KiB — różnice między tymi pomiarami są nieznaczne. Istnieje zatem pewna klasa plików o odpowiednio małym rozmiarze, dla których proces szyfrowania danych stanowi znikomą część całej procedury, a mierzony czas można uznać za stały narzut operacji wykonywanych przez programy wokół samego szyfrowania, dla danej maszyny.



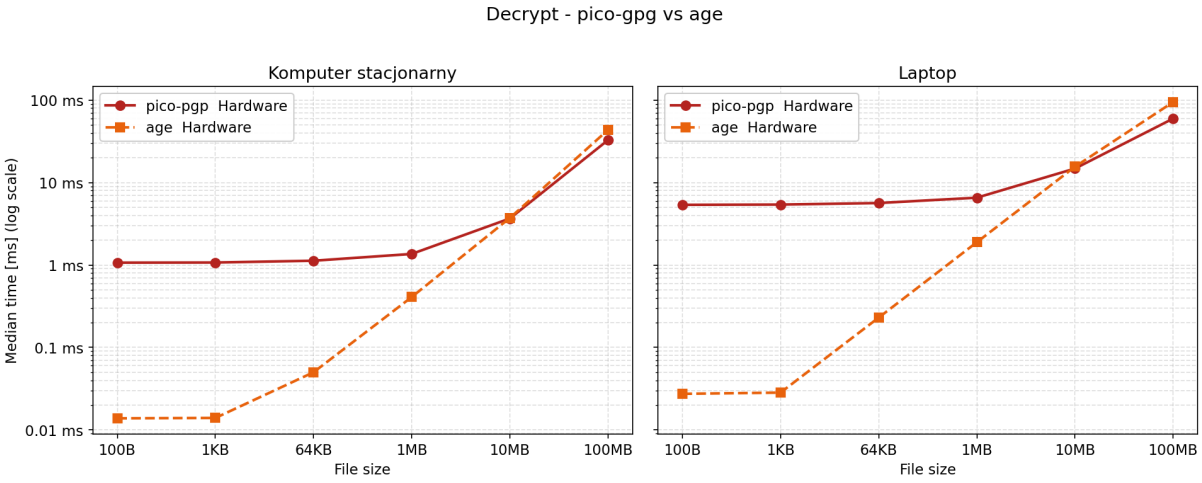
Rysunek 4. Zestawienie wykresów czasu szyfrowania plików za pomocą GPG i age na komputerze stacjonarnym i laptopie

Podobnie jak w przypadku komputera stacjonarnego, wraz ze wzrostem rozmiaru plików rośnie czas potrzebny na ich zaszyfrowanie, a age także na słabszym sprzęcie utrzymuje przewagę szybkości. Ciekawych obserwacji dostarcza porównanie dwóch pierwszych oraz dwóch ostatnich pomiarów. Ponownie widać, że wraz ze wzrostem rozmiaru plików krzywa przyspieszenia między GPG a age się wypłaszcza, tym razem jednak zbiega w okolice pięciokrotnego przyspieszenia, podczas gdy dla komputera stacjonarnego zbiegała ku wartości ok. 3,5. Można stąd wysnuć wniosek, że poziom, na którym zatrzymuje się gwałtowny spadek przyspieszenia, zależy od możliwości obliczeniowych konkretnej maszyny. Podobna obserwacja dotyczy szyfrowania najmniejszych plików: dla laptopa różnica w przyspieszeniu między plikami 100 B a 1 KiB nie jest już tak mała, jak w pomiarach na komputerze stacjonarnym. Wobec tej różnicy trudno mówić o stałym narzucie czasowym operacji wokół procesu szyfrowania, również te operacje silnie zależą zatem od mocy maszyny, na której szyfrowanie jest wykonywane.

Znaczna różnica przy mniejszych rozmiarach plików między GPG a age może wynikać z dodatkowego etapu kompresji, jaki GPG wykonuje przed zaszyfrowaniem pliku. Redukcja rozmiaru jest kosztowna czasowo i dla małych plików stanowi znaczną część całego procesu szyfrowania, wraz ze wzrostem rozmiaru pliku czas ten staje się coraz mniejszą częścią całej procedury, przez co jego znaczenie maleje.

4.1.3 Porównanie czasu potrzebnego na odszyfrowanie w zależności od wielkości pliku

Następnym mierzonym parametrem był czas wykonania operacji deszyfrowania danych przez GPG oraz age. Analogicznie jak w przypadku szyfrowania, pomiarów dokonano dla sześciu rozmiarów plików wymienionych w sekcji 3.5, ponownie wszelkie operacje związane z kluczem sprzętowym wykluczono z fragmentów, w których wykonywano pomiar czasu.



Rysunek 5. Zestawienie wykresów czasu deszyfrowania plików za pomocą GPG i age na komputerze stacjonarnym i laptopie

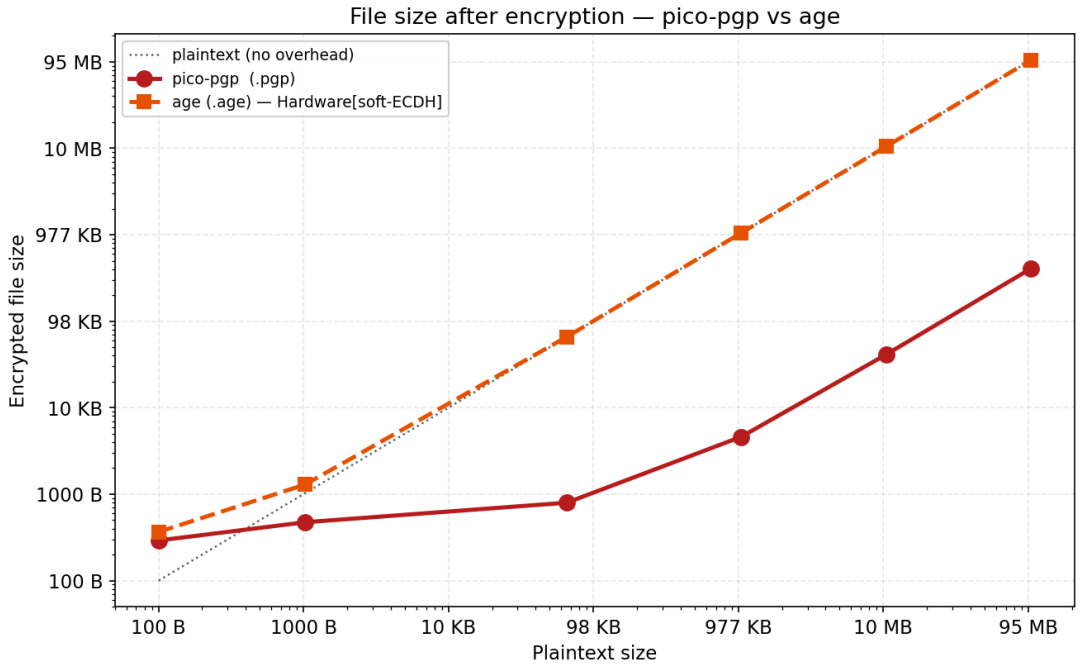
Na Rys. 5 przedstawiono wyniki pomiarów czasu deszyfrowania wykonane na komputerze stacjonarnym oraz laptopie. Omówione zostaną na początku pomiary zebrane dla komputera stacjonarnego. Ponownie, bez zaskoczenia, czas deszyfrowania pliku rośnie wraz ze wzrostem jego rozmiaru. Dla plików o małym rozmiarze widoczna jest ogromna przepaść między deszyfrowaniem GPG a age, przyspieszenie między programami wynosi ponad 76 raza, jednak przewaga age nie utrzymuje się tak, jak miało to miejsce przy szyfrowaniu. Wraz ze wzrostem rozmiaru pliku przyspieszenie drastycznie spada, aż do wyrównania czasów deszyfrowania dla pliku 10 MiB, a dla pliku 100 MiB to GPG wykonuje operację szybciej niż age — 1,33 raza.

Dla maszyny słabszej obliczeniowo trend różnic między GPG a age w operacji deszyfrowania utrzymuje się, a nawet jest bardziej widoczny. Dla mniejszych plików age wykonuje operację 195,8 raza szybciej niż GPG, jednak i tu przyspieszenie zaczyna dramatycznie maleć, aż do wyrównania czasów ponownie dla pliku o rozmiarze 10 MiB. Ciekawą obserwacją jest to, że zarówno dla komputera stacjonarnego, jak i dla laptopa przyspieszenie między programami dla plików od 64 KiB do 100 MiB utrzymuje się na podobnym poziomie i nie zachowuje trendu zaobserwowanego przy szyfrowaniu, gdzie przyspieszenie dla dużych plików było mniejsze dla maszyny o wyższej mocy obliczeniowej.

Dramatyczny spadek przewagi age nad GPG dla większych plików można wyjaśnić krokiem kompresji wykonywanym przez GPG przed szyfrowaniem danych. GPG skompresował 100 MiB do około 407 KiB, deszyfruje więc i dekompresuje jedynie te 407 KiB, podczas gdy age deszyfruje pełne 100 MiB strumienia ChaCha20-Poly1305. Warto zauważyć, że na danych nieściśliwych GPG musiałby przetworzyć pełny rozmiar, przez co wypracowana przewaga zmniejszyłaby się lub zniknęła.

4.1.4 Porównanie wielkości zaszyfrowanych plików

Ponieważ zarówno przy szyfrowaniu, jak i deszyfrowaniu istotny okazał się etap kompresji danych, uznano za zasadne zobrazowanie, jak wielkość szyfrogramu ma się do rozmiaru pliku oryginalnego.



Rysunek 6. Porównanie wielkości szyfrogramów GPG i age względem wagi szyfrowanych plików

Na wykresie 6 zaprezentowano wizualnie dane pomiarowe dotyczące rozmiaru szyfrogramów względem wielkości szyfrowanego tekstu. Z wykresu można odczytać, że jedynym szyfrogramem zauważalnie większym od pliku oryginalnego jest szyfrogram utworzony z pliku o rozmiarze 100 B — GPG wygenerował szyfrogram o wielkości 291 B, a age 300 B. Dla kolejnych plików kompresja GPG jest już widoczna dla pliku 1 KiB pozostaje jeszcze znikoma, jednak od 64 KiB staje się znacząca i postępuje w przybliżeniu liniowo. Jest to oczywiście obserwacja dotycząca wyłącznie plików testowych w przypadku kompresji plików zawierających niewiele powtarzających się elementów byłaby ona znacznie mniej skuteczna. Natomiast dla age kompresja nie występuje, co wyraźnie widać na wykresie: rozmiar szyfrogramu przebiega niemal na równi z wielkością pliku niezaszyfrowanego.

Zamieszczono jedynie jeden wykres z racji, że zarówno dla komputera stacjonarnego, jak i laptopa kompresja wyglądała identycznie.

4.2 Analiza bezpieczeństwa programów szyfrujących

W niniejszej sekcji podjęto analizę udokumentowanych ataków na program GPG oraz sprawdzono, czy najnowszy standard RFC 9580 [42] usuwa istniejące w programie błędy i czy age jest podatny na te same słabości co GPG. W podrozdziałach poniżej każda podatność zostanie scharakteryzowana oraz został zamieszczony opis dotyczący wyników replikacji podatności dla GPG oraz age. Każdą podatność przetestowano na komputerze stacjonarnym, którego specyfikację techniczną opisano w sekcji 3.5. Przed zaprezentowaniem analizy omówiono literaturę, w której opisano testowane w ramach tej pracy podatności.

4.2.1 Omówienie literatury

W ramach etapu przeprowadzania scenariuszy ataków dla obu programów szyfrujących wyselekcjonowano podatności na podstawie luk zlokalizowanych dla PGP w latach ubiegłych. Głównym kryterium wyboru była kwestia, czy dana podatność jest związana z szyfrowaniem plików oraz czy opisywany scenariusz mógłby zostać zrealizowany również za pomocą age. Pierwszą zlokalizowaną podatnością jest eksfiltracja przez kanał CFB (EFail), opisana w pracy „Efail: Breaking S/MIME and OpenPGP Email Encryption using Exfiltration Channels" [27]. Mimo wieku tej pracy opisane w niej słabości protokołu są widoczne po dziś dzień, a brak AEAD w PGP jest eksploatowalny w praktyce, nie tylko teoretycznie. Do podatności wynikających z niedoprecyzowania samego standardu należy interoperacyjny atak na szyfrowanie ElGamala, opisany w pracy „On the (in)security of ElGamal in OpenPGP" [7]. RFC 4880 [10] nie określa jednoznacznie parametrów ElGamala, wskutek czego poszczególne implementacje przyjęły odmienne konwencje ich generowania. Choć każda z nich rozpatrywana osobno pozostaje bezpieczna, w OpenPGP ich zestawienie umożliwia atak typu cross-configuration, prowadzący do odtworzenia tekstu jawnego, a nawet klucza prywatnego, z pojedynczego szyfrogramu. W dalszej kolejności uwzględniono podatności wynikające z zastosowania przestarzałych algorytmów blokowych. Praca „On the Practical (In-)Security of 64-bit Block Ciphers" [3] stanowi klasyczny dowód, że szyfr o 64-bitowym bloku, a takim jest wymagany przez standard OpenPGP algorytm 3DES, jest praktycznie niebezpieczny przy dużych wolumenach danych z uwagi na atak urodzinowy na kolizje bloków.

Odrębną klasę stanowią podatności architektoniczne samego formatu klucza. Praca „Victory by KO: Attacking OpenPGP Using Key Overwriting" [4] wykazuje, że pakiet Secret Key Packet nie wiąże kryptograficznie pól publicznych z prywatnymi, co umożliwia podstawienie międzyalgorytmiczne (cross-algorithm substitution) i w konsekwencji pełne odtworzenie klucza prywatnego.

Kolejną analizowaną słabością kryptograficzną jest funkcja wyprowadzania klucza (S2K) oparta na SHA-1. Standard OpenPGP definiuje iterowany i solony S2K na SHA-1 [42, §3.7.1], mimo że SHA-1 zaprojektowano z myślą o szybkości, a nie o zastosowaniu w KDF: układy GPU obliczają miliardy wartości SHA-1 na sekundę, a brak twardości pamięciowej (memory-hardness) pozwala w pełni wykorzystać ich moc w ataku offline. Słabość ta jest dokładnie opisana w pracy "Stronger Key Derivation via Sequential Memory-Hard Functions" [26]. Słabość SHA-1 nie ogranicza się do jej szybkości. Funkcja ta jest w formacie v1 SEIPD jedyną dopuszczoną podstawą kodu wykrywania modyfikacji (MDC), zapewniającego integralność zaszyfrowanej wiadomości, mimo że jej odporność na kolizje została praktycznie obalona. W pracy „SHA-1 is a Shambles" [17], autorzy zademonstrowali atak personifikacyjny wprost na sieć zaufania PGP. Teoretycznie pozwala to na taką modyfikację szyfrogramu, aby zmieniony tekst jawny zachował zgodny skrót MDC i przeszedł kontrolę integralności.

Kolejna grupa ataków związana jest z samym sposobem, w jaki PGP szyfruje pliki. Praca „Reducing Metadata Leakage from Encrypted Files and Communication with PURBs" [24] wykazuje, że większość formatów szyfrowanych ujawnia metadane poprzez jawne nagłówki, wersję formatu,

użyte schematy, liczbę odbiorców, a nawet ich tożsamości, co pozwala m.in. ujawnić pełną listę współpracowników na podstawie jednej wiadomości e-mail lub zidentyfikować wersję oprogramowania nadawcy. Następna podatność wynika z faktu, że PGP kompresuje dane przed operacją szyfrowania; problem ten został opisany w pracy „Compression and Information Leakage of Plaintext" [16].

Osobną kategorię stanowi podatność na błędy konfiguracji. Praca „SoK: Why Johnny Can't Fix PGP Standardization" [12] wskazuje, że mnogość opcjonalnych algorytmów i flag, czyni go podatnym na błędną konfigurację. Ostatnia klasa ataków związana jest ze złożonością protokołu PGP. Praca „Security Applications of Formal Language Theory" [29] dostarcza teoretycznych podstaw do traktowania złożoności parsera jako miary powierzchni ataku, dowodzi, że każdy wzrost złożoności protokołu należy traktować jako zwiększenie ryzyka. Autorzy formalizują też technikę parse tree differential attack, w której dwa moduły interpretują ten sam strumień bajtów odmiennie, co można zaobserwować dla persera OpenPGP.

4.2.2 Podatność: EFail / CFB Malleability

Tryb CFB [8], którym OpenPGP szyfruje ładunek wiadomości, ma właściwość *malleability*: zmiana bajtu szyfrogramu o wartość δ (XOR) zmienia odpowiadający mu bajt tekstu jawnego po odszyfrowaniu o tę samą wartość δ. Zatem kontrola nad szyfrogramem bez znajomości klucza daje kontrolę nad fragmentem odszyfrowanej treści. Atak EFail (2018 r.) [27] wykorzystał to, otaczając przechwycony, zaszyfrowany tekst spreparowanym znacznikiem HTML (`<img src="http://atak.pl/SEKRET">`); klient pocztowy ofiary odszyfrowywał i renderował HTML, wysyłając odszyfrowany sekret na serwer atakującego bez łamania samego szyfrowania. RFC 9580 (2024 r.) wprowadziło uwierzytelnione SEIPD v2 [42, §5.13.2], ale dla kompatybilności wstecznej zachowało podatny SEIPD v1 [42, §5.13.1] - słabość formalnie pozostaje w standardzie.

Replikacja tego ataku dla GPG polegała na zaszyfrowaniu wiadomość `TAJNY DOKUMENT FINANSOWY Przelew: 50000 PLN Konto docelowe: PL61109010140000071219812874 Autoryzacja: CONFIRMED` symetrycznie z trybem bez ochrony integralności (flagi `--symmetric --rfc2440`). Następnie zmodyfikowano 50 bit w szyfrogramie. Po odszyfrowaniu tylko fragment sąsiadujący z modyfikacją stał się nieczytelny, ale numer konta i kwota pozostały w pełni czytelne mimo ingerencji bez znajomości klucza. GPG zgłasza to kodem wyjścia 2 z komunikatem `wymuszono błąd odszyfrowywania!`, lecz zmodyfikowana treść może trafić na standardowe wyjście niezależnie od tego ostrzeżenia. Tę lukę wykorzystał EFail. Identyczny test na pliku zaszyfrowanym za pomocą age zakończył się odmową zwrócenia jakichkolwiek danych, kod wyjścia 1, `no identity matched any of the recipients`. Weryfikacja tagu Poly1305 [39], bezpieczny IND-CCA2 [2], odbywa się przed zwróceniem tekstu jawnego, więc modyfikacja unieważnia deszyfrowanie w całości.

Podatność jest architektoniczna, nie implementacyjna, jej usunięcie wymagałoby porzucenia SEIPD v1, więc mimo ostrzeżeń pozostaje dostępna w GPG.

4.2.3 Podatność: Adaptive Chosen-Ciphertext Attack

W klasycznych schematach kryptograficznych ataki z adaptacyjnie dobieranym szyfrogramem polegają na modyfikowaniu przechwyconego szyfrogramu i obserwowaniu reakcji wyroczni w celu stopniowego odzyskiwania tekstu jawnego [23]. W przypadku PGP wektor takiego ataku wynika ze specyficznej modyfikacji standardowego trybu CFB [42, §13.4]. Standard PGP nie przesyła wektora inicjalizującego w postaci jawnej, zamiast tego na początku procesu szyfrowania generowany jest losowy blok o rozmiarze bloku szyfru 8 bajtów dla 3DES lub 16 bajtów dla AES. Do szyfrogramu dołączana jest dwubajtowa redundancja, dzięki temu odbiorca może szybko zweryfikować poprawność podanego hasła lub klucza sesyjnego przed przystąpieniem do deszyfracji całego pliku. Ten mechanizm nazywa się *quick check*.

Mechanizm ten stanowi nieautoryzowaną kontrolę integralności na niezapewniającym uwierzytelnienia trybie szyfrowania, co tworzy podatność na ataki typu oracle [23]. Atakujący wykorzystuje znajomość przewidywalnego początku pliku, aby poprzez odpytywanie wyroczni i manipulację dwubajtowym polem sprawdzającym ustalić stan kryptograficzny powiązany z kluczem sesyjnym. Następnie podstawia docelowe bloki szyfrogramu na początek wiadomości, co przy średnio 32 tysiącach zapytań na blok pozwala bezpośrednio odczytać pierwsze dwa bajty każdego z nich bez znajomości klucza.

Proces udokumentowania podatności dla GPG rozpoczął się od zaszyfrowania wiadomości (`"TAJNY DOKUMENT: klucz dostepu XK-7734-ALFA"`) symetrycznie, przy użyciu AES128 oraz bez kompresji używając odpowiednich flag: `--symmetric --rfc2440 --cipher-algo AES128 --compress-algo none`. W przeprowadzanej replikacji zasymulowano działanie wyroczni zwracającej wyłącznie sukces lub porażkę dla *quick check*, bez dostępu do treści odkodowywanej wiadomości. W ramach ataku przeprowadzono oba etapy: fazę ustalenia stałej klucza sesyjnego zakończono po 57 732 zapytaniach (≈62 min), fazę właściwego ataku na kolejny blok po 40 759 zapytaniach (≈44 min), odzyskując poprawnie oba bajty bloku M_2 (`'KU'`) przy łącznym koszcie 98 493 zapytań w ≈106 minut (rzeczywista przepustowość ∼15 zapytań/s).

Analogiczny plik zaszyfrowany przy pomocy age po identycznej modyfikacji bajtu szyfrogramu zwrócił kod wyjścia 1 i komunikat `age: error: no identity matched any of the recipients`, bez zwrócenia jakiegokolwiek fragmentu danych.

4.2.4 Podatność: SWEET32 - atak urodzinowy na 3DES

Atak SWEET32 wykorzystuje ograniczenia bezpieczeństwa szyfrów blokowych o małym, 64-bitowym rozmiarze bloku, takich jak 3DES, wynikające z paradoksu urodzinowego [3]. Dla szyfru o bloku n-bitowym kolizja dwóch bloków szyfrogramu staje się wysoce prawdopodobna już po zaszyfrowaniu około $2^{\mathrm{n}/2}$ bloków. W przypadku 64-bitowego 3DES próg ten wynosi 2^{32} bloków, czyli zaledwie ok. 32 GB przesłanych danych w ramach jednej sesji. Po osiągnięciu 32 GB prawdopodobieństwo wystąpienia kolizji przekracza 39%, a przy 40 GB wynosi już ponad 50%.

W trybie CFB, gdzie i-ty blok szyfrogramu wyliczany jest jako: $\mathrm{C_i} = \mathrm{E_K}(\mathrm{C_{i-1}}) \oplus \mathrm{P_i}$. W sytuacji, gdy w wygenerowanym strumieniu danych dojdzie do kolizji dwóch pełnych bloków szyfrogramu ($\mathrm{C_i} = \mathrm{C_j}$), strumień klucza generowany dla kolejnych bloków staje się identyczny: $\mathrm{E_K}(\mathrm{C_i}) = \mathrm{E_K}(\mathrm{C_j})$. W konsekwencji różnica XOR kolejnych bloków szyfrogramu odpowiada bezpośrednio różnicy XOR kolejnych bloków tekstu jawnego: $\mathrm{C_{i+1}} \oplus \mathrm{C_{j+1}} = (\mathrm{E_K}(\mathrm{C_i}) \oplus \mathrm{P_{i+1}}) \oplus (\mathrm{E_K}(\mathrm{C_j}) \oplus \mathrm{P_{j+1}}) = \mathrm{P_{i+1}} \oplus \mathrm{P_{j+1}}$. Jeżeli atakujący zna treść jednego z bloków tekstu jawnego $\mathrm{P_{i+1}}$ (np. powtarzalny nagłówek pakietu danych), może w pełni odzyskać nieznany blok $\mathrm{P_{j+1}}$ wyłącznie na podstawie podsłuchanego szyfrogramu: $\mathrm{P_{j+1}} = \mathrm{P_{i+1}} \oplus \mathrm{C_{i+1}} \oplus \mathrm{C_{j+1}}$.

W ramach badań przeprowadzono próbę replikacji podatność szyfrów o 64-bitowym bloku na atak kolizyjny SWEET32. Aby wymusić użycie algorytmu 3DES przez GPG, zaszyfrowano plik testowy z użyciem flagi wymuszającej użycie konkretnego algorytmu `--cipher-algo 3DES`. Inspekcja struktury pakietów za pomocą komendy gpg `--list-packets` potwierdziła użycie trybu `3DES.CFB`. Pełne wygenerowanie kolizji 64-bitowej wymaga przesłania 32–40 GB danych pod jednym kluczem sesyjnym. Z tego względu demonstrację podzielono na dwa komplementarne etapy.

Pierwszy etap miał służyć empirycznemu potwierdzeniu zaistnienia paradoksu urodzinowego. W celu potwierdzenia właściwości statystycznych szyfrogramu wygenerowano 128 KB danych. Przeprowadzono analizę kolizji na zredukowanej przestrzeni prefiksów (3 bajty / 24 bity). Zgodnie z przewidywaniami teoretycznymi, w badanym zbiorze zidentyfikowano powtarzające się wartości, m.in. dla bloków $\mathrm{i} = 3634$ oraz $\mathrm{j} = 7046$:

```
C[3634] = 7d1b8f9897dc4720
C[7046] = 7d1b8f7cf6271b4b  -->  C[3634][:3] == C[7046][:3] == 0x7d1b8f
```

W drugim etapie zweryfikowano algorytmiczną poprawność odzyskiwania sekretu przy zaistnieniu pełnej kolizji bloków $\mathrm{C_2} = \mathrm{C_5} =$ `0xadcbdeb0e217e7d8`. Przyjmując znajomość bloku $\mathrm{P_3}$, wykonano operację $\mathrm{P_6} = \mathrm{P_3} \oplus \mathrm{C_3} \oplus \mathrm{C_6}$:

```
P[3] XOR P[6] = 8a58a3ba630e6e35  (nieznana relacja jawna)
C[3] XOR C[6] = 8a58a3ba630e6e35  (wartość obserwowana w szyfrogramie)
Odzyskany blok P[6]: 0xd91de0e8265a4f14 (zgodność: 100%)
```

Eksperyment potwierdził, że wystąpienie kolizji w trybie CFB natychmiastowo kompromituje poufność sąsiadującego bloku danych.

Próby replikacji podatności na age nie były możliwe, age nie obsługuje szyfrowania plików za pomocą 3DES używa szyfrowania strumieniowego (ChaCha20) wiec nie ma bloków na których można by przeprowadzić atak z wykorzystaneim paradoksu urodzinowego. Próg urodzinowy dla age dotyczy przestrzeni nonce (2^{96}) i aby był on skuteczny wymagałby 2^{48} wiadomości zaszyfrowanych tym samym kluczem.

4.2.5 Podatność: SHA-1 S2K - brute-force offline hasła

Jednym z kluczowych zadań procedur wyprowadzania klucza z hasła jest sztuczne wydłużenie czasu weryfikacji pojedynczej próby [26]. Ma to na celu uniemożliwienie lub drastyczne podniesienie kosztu ataków słownikowych i brutalnej siły przeprowadzanych po przechwyceniu zaszyfrowanego pliku.

PGP może wymusić (we wcześniejszych implementacjach było to ustawienie domyślne) użycie funkcji skrótu SHA-1 w operacji *String-to-Key* [10, §3.7.1.3]. Algorytm SHA-1 zaprojektowano pod kątem maksymalizacji szybkości przetwarzania przy minimalnym zapotrzebowaniu na pamięć operacyjną. Cecha ta stanowi krytyczną wadę w kontekście bezpieczeństwa haseł. Ze względu na znikome zapotrzebowanie SHA-1 na pamięć RAM, architektura GPU pozwala na pełne, równoległe wysycenie rdzeni obliczeniowych w trakcie próby odnalezienia szukanego hashu hasła [26].

W celu zmierzenia realnej przewagi obliczeniowej atakującego dysponującego nowoczesną kartą graficzną przeprowadzono testy porównawcze przy użyciu oprogramowania `hashcat 7.1.2` [32] na karcie NVIDIA GeForce RTX 5070 (12 GB VRAM, środowisko OpenCL 3.0). Na początku wygenerowano dwa zaszyfrowane pliki testowe z identycznym hasłem bazowym: dla GPG z wymuszonym algorytmem SHA-1 do funkcji *S2K* oraz za pomocą age z użyciem hasła użytkownika jako element zabezpieczający.

Wydajność przeszukiwania przestrzeni kluczy zmierzono w dedykowanych trybach benchmarku `hashcat`: tryb 17010 - modelujący łamanie hasła dla pakietu symetrycznego GPG oraz tryb 8900 - modelujący atak na sekwencyjną funkcję pamięciożerną używaną w formacie age.

W testowanym środowisku karta graficzna osiągnęła przepustowość ponad 13,18 mln operacji na sekundę dla schematu SHA-1 S2K stosowanego w GPG, co oznacza czas weryfikacji pojedynczego hasła na poziomie zaledwie 75,83 ns. Dla porównania, w przypadku funkcji *scrypt* przepustowość spadła do 3,703 H/s (czas weryfikacji: ok. 0,27 ms). Różnica wydajnościowa wyniosła około 3,561 raza.

Aby zilustrować praktyczne konsekwencje tej dysproporcji, oszacowano czas potrzebny na wyczerpujące przeszukanie popularnego słownika haseł `rockyou.txt`. Dla GPG przeszukanie całego słownika zajmuje zaledwie ok. 1,09 sekundy. Natomiast dla age przeszukanie tego samego słownika na identycznym sprzęcie wymaga ok. 65 minut.

4.2.6 Podatność: Metadata Leak - ujawnienie odbiorcy przez Key ID

Bezpieczeństwo protokołów szyfrowania asymetrycznego tradycyjnie oceniano przez pryzmat poufności samego ładunku. Współczesna kryptoanaliza kładzie jednak równorzędny nacisk na ochronę metadanych, formalizowaną jako cecha prywatności klucza [24]. Zgodnie z tą definicją, pasywny obserwator przechwytujący szyfrogram nie powinien być w stanie ustalić, do kogo wiadomość została zaadresowana, ani jakim kluczem publicznym została zaszyfrowana.

W standardzie PGP ([42, §5.1.1]) aby odbiorca posiadający wiele kluczy prywatnych nie musiał wykonywać deszyfracji każdego pakietu, nagłówek PKESK zawiera jawne, niezaszyfrowane 64-bitowe pole *Key ID*. Wartość tego pola stanowi najmniej znaczące 8 bajtów odcisku palca klucza publicznego

odbiorcy [42, §5.5.4]. Klucze publiczne w ekosystemie PGP są z definicji jawne i dystrybuowane za pośrednictwem publicznych serwerów kluczy. W rezultacie każdy podsłuchujący węzeł pośredniczący w transmisji może bez znajomości kluczy prywatnych jednoznacznie zidentyfikować strony komunikacji, co stanowi bezpośredni wektor ataków [24].

W celu zademonstrowania podatności wygenerowano 2048-bitową parę kluczy RSA dla tożsamości `Alicja Karp <alicja@example.com>`, uzyskując klucz główny `E7C05AB326F3AF99` oraz podklucz szyfrujący `64FA0EDE0C94B3A6`. Następnie zaszyfrowano poufny dokument tekstowy. Symulując przechwycenie pliku przez nieuprawnionego obserwatora, wykonano inspekcję struktury pakietów za pomocą komnedy `gpg --list-packets`:

```
gpg: zaszyfrowano kluczem RSA o identyfikatorze 64FA0EDE0C94B3A6
gpg: błąd odszyfrowywania kluczem publicznym: Brak klucza tajnego
gpg: błąd odszyfrowywania: Brak klucza tajnego
# off=0 ctb=85 tag=1 hlen=3 plen=268
:pubkey enc packet: version 3, algo 1, keyid 64FA0EDE0C94B3A6
        data: [2047 bits]
# off=271 ctb=d2 tag=18 hlen=2 plen=165 new-ctb
:encrypted data packet:
        length: 165
        mdc_method: 2
```

Kolejnym krokiem atakującego było skorelowanie pozyskanego identyfikatora z ogólnodostępną bazą kluczy publicznych np. za pomocą komendy `gpg --list-keys --keyid-format long`:

```
gpg: sprawdzanie bazy zaufania
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: poziom: 0 poprawnych:   3 podpisanych:   0 zaufanie: 0-,0q,0n,0m,0f,3u
[keyboxd]
---------
pub   rsa2048/E7C05AB326F3AF99 2026-05-31 [SCEAR]
      6BC5613D12E89DFDC59B8DD1E7C05AB326F3AF99
uid           [   absolutne   ] Alicja Karp <alicja@example.com>
sub   rsa2048/64FA0EDE0C94B3A6 2026-05-31 [SEA]
...
```

Identyfikator `64FA0EDE0C94B3A6` jednoznacznie wskazał podklucz Alicji Karp. Atakujący bez łamania kryptografii i bez dostępu do klucza prywatnego uzyskał 100% pewności co do tożsamości odbiorcy dokumentu.

W analogicznym scenariuszu przetestowano narzędzie age. Wygenerowano klucz odbiorcy w standardzie X25519 i zaszyfrowano plik źródłowy. Inspekcja nagłówka pliku wykazała następującą strukturę:

```
age-encryption.org/v1
```

```
-> X25519 uS4vvTckdMafkQVRakDTh8KtXGTP2yeAl6ExTTjMWQ0
zc5nuvVuJoJFdX5wgywCchIvQHB7/U70AivpmlXvPcE
--- SbFlsQjN4HUBpnmYHEqiH3xNlwQuNsDd7NohlYTzfzQ
```

Format age całkowicie eliminuje długoterminowe identyfikatory. W nagłówku umieszczany jest wyłącznie jednorazowy, efemeryczny klucz publiczny X25519, generowany losowo dla danej operacji szyfrowania. Punkt ten nie występuje w żadnej publicznej bazie i nie posiada żadnego matematycznego powiązania z tożsamością ani kluczem publicznym Alicji Karp.

4.2.7 Podatność: Compression Oracle

Większość bezstratnych algorytmów kompresji działa na zasadzie wyszukiwania i usuwania powtarzających się ciągów bajtów. Gdy algorytm napotka sekwencję, która wystąpiła wcześniej w przetwarzanym tekście, zastępuje ją krótkim wskaźnikiem. Ta cecha algorytmów kompresyjnych, opartych na rozmiarze skompresowanego strumienia danych, została zakwalifikowana jako kanał boczny do przeprowadzania ataków [16]. Kanał boczny powstaje w sytuacji, gdy w jednym przetwarzanym dokumencie znajdują się obok siebie dane poufne oraz dane kontrolowane przez atakującego. Jeżeli atakujący wstrzyknie ciąg znaków identyczny z fragmentem sekretu, kompresor potraktuje go jako duplikat, co spowoduje lepszą kompresję i zauważalny spadek rozmiaru wyjściowego. W przypadku błędnego zgadnięcia nowy ciąg zostanie potraktowany jako unikalny, zwiększając rozmiar pliku wyjściowego.

Zgodnie ze specyfikacją PGP ([42, §2.1]), implementacje standardu powinny domyślnie kompresować dane po złożeniu podpisu, lecz bezpośrednio przed ich zaszyfrowaniem. Ponieważ stosowane tryby szyfrowania symetrycznego zachowują lub ściśle odzwierciedlają długość tekstu wejściowego, rozmiar wynikowego szyfrogramu bezpośrednio ujawnia stopień kompresji danych [16].

W celu zademonstrowania podatności zaimplementowano scenariusz symulujący środowisko ciągłej integracji (CI/CD), w którym usługa automatycznie archiwizuje i szyfruje logi systemowe zawierające poufny nagłówek autoryzacyjny - `Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.SECRET_TOKEN_ABCDEF`. Założono, że atakujący może wstrzykiwać własny tekst do logu (np. poprzez metadane użytkownika wyzwalającego zadanie), a następnie obserwuje wyłącznie całkowity rozmiar wygenerowanego szyfrogramu.

Do weryfikację podatności dla GPG użyto domyślną kompresją ZIP. Skrypt atakujący wstrzykiwał kolejne hipotezy znakowe, monitorując zmiany rozmiaru szyfrogramu:

```
Rozmiar ciphertextu bez wstrzykniecia: 146B
  Pozycja  1: 'S' (148B) | zgadnieto: 'S'
  Pozycja  2: 'E' (148B) | zgadnieto: 'SE'
  Pozycja  3: 'C' (148B) | zgadnieto: 'SEC'
  ...
  Pozycja 18: 'E' (148B) | zgadnieto: 'SECRET_TOKEN_ABCDE'
  Pozycja 19: 'F' (147B) | zgadnieto: 'SECRET_TOKEN_ABCDEF'
```

Dzięki lepszej kompresji przy prawidłowym dopasowaniu prefiksu, atakujący bez znajomości klucza odtworzył pełną wartość 19-znakowego tokenu `SECRET_TOKEN_ABCDEF` w zaledwie 19 krokach.

Identyczny eksperyment przeprowadzono wobec narzędzia age. Wynik próby ataku zaprezentowano poniżej:

```
Rozmiar ciphertextu bez wstrzykniecia: 276B
  Pozycja 1: min=298B max=298B delta=0B
  Pozycja 2: min=298B max=298B delta=0B
  Pozycja 3: min=298B max=298B delta=0B
```

W przypadku age zmiana wstrzykiwanego tekstu nie wywołała żadnych zmian rozmiaru szyfrogramu. Wyrocznia nie dostarczyła żadnego sygnału korelacyjnego, uniemożliwiając przeprowadzenie ataku.

4.2.8 Podatność: Parser Attack Surface

Jednym z fundamentalnych założeń bezpieczeństwa opartego na teorii języków formalnych jest zasada, że parser protokołu stanowi pierwszą linię obrony i zarazem główną powierzchnię ataku systemu [29]. Im bardziej złożona, niejednoznaczna i wielowariantowa jest gramatyka formatu danych, tym większa liczba gałęzi decyzyjnych musi zostać zaimplementowana w kodzie źródłowym parsera, co wprost proporcjonalnie zwiększa prawdopodobieństwo wystąpienia krytycznych błędów logicznych oraz uszkodzeń pamięci [12, 29].

W celu empirycznego porównania odporności obu parserów dla GPG i age przeprowadzono serię testów zniekształcających oraz zbadano anomalie semantyczne wynikające ze specyfikacji.

Testy odpornościowe polegające na systematycznej mutacji każdego bitu i offsetu oraz obcinaniu zaszyfrowanych plików na każdej długości zwróciły następujące dane: dla GPG zarejestrowano 3651 unikalnych błędów wykonania, które sklasyfikowano do 10 odrębnych gałęzi logicznych obsługi błędów parsera, natomiast dla age test ujawnił jedynie 6 powtarzalnych sygnatur błędów, reprezentujących zaledwie 5 gałęzi logiki.

Drugim elementem, który zbadano był mechanizm fragmentarycznego kodowania długości, który umożliwia zakodowanie identycznej zawartości na wiele różnych sposobów na poziomie bajtowym. Dla GPG wygenerowano dwa pliki o tej samej treści tekstowej jeden z jawną, stałą długością nagłówka oraz drugi z kodowaniem fragmentarycznym. Polecenie dekompresji i ekstrakcji, `gpg -d`, dla obu próbek zwróciło bajtowo identyczny wynik. Brak kanonicznej reprezentacji w formacie PGP jest źródłem zjawiska *parse tree differential* [29] – sytuacji, w której dwa niezależne moduły budują różne drzewa rozbioru tego samego pliku. Dla age taki test jest niemożliwy do wykonania, gdyż ma on strukturę ścisła i nie jest możliwe zjawisko niekanoniczności reprezentacji.

Ostatnim zbadanym zjawiskiem była amplifikacja zasobów podczas parsowania. GPG podczas operacji pasywnego listowania struktury pakietów wykonuje pełną dekompresję napotkanych danych jeszcze przed fazą właściwej weryfikacji kryptograficznej. Wykorzystując możliwość kaskadowego zagnieżdżania pakietów skompresowanych, spreparowano zminiaturyzowany plik wejściowy. W trakcie samego parsowania nagłówków wskaźnik amplifikacji rozmiaru osiągnął wartość 410,3:1, co dowodzi,

że podatność typu *zip-bomb* w formacie PGP może zostać wyzwolona na poziomie samego parsera bez znajomości kluczy deszyfrujących [42, §13.14]. Ponownie replikacja dla age nie była możliwa z racji, że oprogramowanie to nie daje możliwości kompresji przy szyfrowaniu.

4.2.9 Podatność na błędną konfigurację

Jedną z fundamentalnych cech architektury PGP jest tzw. zwinność kryptograficzna [12]. W założeniu podejście to miało umożliwić płynną wymianę pojedynczych algorytmów (szyfrów symetrycznych, trybów pracy, funkcji skrótu, procedur S2K) w miarę ich starzenia się i postępów kryptoanalizy. W praktyce w inżynierii bezpieczeństwa nadmierna elastyczność okazała się jednak poważnym wektorem zagrożeń, przenoszącym ciężar oceny bezpieczeństwa na użytkownika lub programistę integrującego narzędzie [12].

Problem architektury PGP wynika ze sztywnych wymogów interoperacyjności nakładanych przez kolejne specyfikacje standardu ([10, §9], [42, §9.3, §13.1]). W przeciwieństwie do takiego podejścia, twórcy narzędzia age przyjęli paradygmat kryptografii zorientowanej oraz odporności na niewłaściwe użycie [37, 39]. Zgodnie z tą filozofią, każdy udostępniony użytkownikowi parametr konfiguracyjny stanowi potencjalne źródło błędu ludzkiego lub celowej manipulacji.

W środowisku testowym przeprowadzono serię prób wymuszenia skompromitowanych algorytmów kryptograficznych podczas operacji szyfrowania dla GPG i age. Domyślna konfiguracja programu GPG stosuje bezpieczne szyfrowanie symetryczne np. AES256.CFB. W ramach testu zweryfikowano podatność procesu na wymuszenie słabych parametrów za pomocą odpowiednich flag:

1. wymuszenie 64-bitowego szyfru 3DES za pomocą falg `--cipher-algo 3DES --allow-old-cipher-algos`
2. wymuszenie funkcji skrótu SHA-1 w procedurze KDF za pomocą flag `--s2k-digest-algo SHA1 --allow-old-cipher-algos`
3. wymuszenie kombinacji algorytmów CAST5 i MD5 za pomocą flag `--cipher-algo CAST5 --digest-algo MD5 --allow-old-cipher-algos`

W żadnym z powyższych przypadków oprogramowanie GPG nie zablokowało operacji ani nie zwróciło kodu błędu. Ewentualne ostrzeżenia trafiają wyłącznie na standardowy strumień błędów, przez co w przypadku zautomatyzowanych skryptów atakujący dysponujący dostępem do parametrów wywołania może w sposób niezauważalny obniżyć poziom bezpieczeństwa szyfrowanych danych.

Dla age przeprowadzono analizę interfejsu wiersza poleceń narzędzia:

```
$ age --help
Usage:
    age [--encrypt] (-r RECIPIENT | -R PATH)... [--armor] [-o OUTPUT] [INPUT]
    age [--encrypt] --passphrase [--armor] [-o OUTPUT] [INPUT]
    age --decrypt [-i PATH]... [-o OUTPUT] [INPUT]
```

age nie posiada żadnych flag umożliwiających wybór szyfru symetrycznego, trybu pracy, funkcji skrótu czy parametrów KDF. Do szyfrowania asymetrycznego zawsze stosowana jest kombinacja X25519 wraz z ChaCha20-Poly1305, do szyfrowania hasłem obligatoryjnie wykorzystywana jest funkcja scrypt, a brak mechanizmu negocjacji wyklucza ataki typu *downgrade* oraz błędy konfiguracyjne.

4.2.10 Ataki niezreplikowane

Spośród przeanalizowanych podatności jedynie trzech nie udało się prawidłowo zreplikować dla ustalonego środowiska (przyjętych wersji GPG i age oraz mocy obliczeniowej używanej maszyny). Były to: *downgrade* szyfrowania uwierzytelnionego (AEAD) do trybu CFB, interoperacyjny atak na ElGamala oraz sfałszowanie kodu wykrywania modyfikacji z użyciem kolizji SHA-1. Przyczyny niepowodzenia są nieco inne dla każdej z tych podatności, jednak z uwagi na małą liczbę takich przypadków postanowiono opisać je w ramach jednej sekcji.

Zaczynając od podatności na *downgrade* szyfrowania AEAD do trybu CFB, wykorzystuje ona fakt, że implementacja obsługująca nowoczesne szyfrowanie uwierzytelnione musi, ze względu na zgodność wsteczną, nadal odczytywać starsze pakiety oparte na trybie CFB (SEIPD v1, [42, §5.13.1]) oraz przestarzały pakiet SED. Atakujący przekształca pakiet AEAD tak, aby odbiorca odszyfrował go jako dane w trybie CFB, sprowadzając problem do znanej podatności na *malleability* oraz do wyroczni deszyfrującej. Replikacji nie przeprowadzono, ponieważ zastosowana wersja GnuPG (2.4.9) zawiera już mechanizmy blokujące ten wariant degradacji, a jego wywołanie wymagałoby celowego cofnięcia się do wersji sprzed wprowadzenia poprawki, co wykraczałoby poza zakres badania współczesnego oprogramowania.

Kolejną podatnością było wykorzystanie słabości szyfrowania ElGamal. Autorzy pracy „On the (In)Security of ElGamal in OpenPGP" [7] wykazali, że niedoprecyzowanie parametrów szyfrowania ElGamala w RFC 4880 [10] prowadzi do sytuacji, w której dwie samodzielnie bezpieczne implementacje, komunikując się poprzez format OpenPGP, tworzą konfigurację podatną na rozwiązanie problemu logarytmu dyskretnego i odzyskanie klucza sesyjnego. Replikacja nie była możliwa, ponieważ atak wymaga jednoczesnego zestawienia dwóch różnych i wzajemnie niekompatybilnych implementacji ElGamala, z których przynajmniej jedna odpowiada podatnej, wycofanej z użycia wersji (np. Botan [19] w wersji wcześniejszej niż 2.17). Środowisko badawcze opiera się na bibliotece libgcrypt, która generuje bezpieczne parametry, wobec czego odtworzenie ataku wymagałoby zbudowania i utrzymania archiwalnej, niewspieranej już biblioteki wyłącznie w celu wywołania rozbieżności konfiguracyjnej. Dodatkowo RFC 9580 [42] rozwiązuje problem swojego poprzednika, wprowadzając nowy standard, który nie zalicza ElGamala do algorytmów zalecanych, a klucze w wersji 6 opierają się na jednoznacznie wyspecyfikowanych krzywych eliptycznych (np. X25519 czy Ed25519), przez co pierwotna przyczyna, niedookreślenie parametrów, przestaje występować. Warto jednak zauważyć, że ElGamal pozostaje w rejestrze algorytmów ze względu na zgodność ze starszymi kluczami.

Ostatnią podatnością, której nie udało się zweryfikować, jest możliwość fałszowania MDC z użyciem kolizji SHA-1. Kod wykrywania modyfikacji w formacie SEIPD v1 opiera integralność

zaszyfrowanej wiadomości na funkcji skrótu SHA-1, obliczanej nad tekstem jawnym poprzedzonym losowym prefiksem. Teoretyczna podatność zakłada wykorzystanie znanych kolizji SHA-1 [17] do takiej modyfikacji zaszyfrowanej wiadomości, aby zmieniony tekst jawny dawał ten sam skrót MDC, a więc przechodził kontrolę integralności. Problem z wykazaniem tej podatności dla GPG pojawił się z dwóch przyczyn. Po pierwsze, dopasowanie kolizji do dwóch wiadomości o różnych prefiksach wymaga kolizji o dobranym prefiksie i dużym nakładzie mocy obliczeniowej. Przy przepustowości pojedynczej karty NVIDIA GeForce RTX 5070, wynoszącej rząd kilkunastu gigahashy SHA-1 na sekundę, wygenerowanie świeżej kolizji tej klasy trwałoby rzędu dziesięcioleci; dla porównania, oryginalne ataki zrealizowano na dużych klastrach GPU. Po drugie, pomijając już sam koszt, format uniemożliwia proste wklejenie gotowej kolizji. Tryb CFB w PGP poprzedza tekst jawny losowym prefiksem o długości bloku szyfru powiększonej o dwa oktety, generowanym przez stronę szyfrującą w chwili szyfrowania. Ponieważ dane wejściowe funkcji SHA-1 rozpoczynają się od tego niekontrolowanego przez napastnika prefiksu, warunek wspólnego prefiksu nie może zostać spełniony, a bloki kolizyjne nie dają się umieścić w wymaganym położeniu. Dodatkowo propagacja błędu w trybie CFB sprawia, że modyfikacja bloku szyfrogramu niszczy blok następny, co jeszcze bardziej utrudnia kontrolowane rozmieszczenie bloków kolizyjnych w domenie tekstu jawnego.

Zestawienie przyczyn niepowodzenia pokazuje, że żadna z trzech klas nie została zablokowana przez wadę w metodyce badania, lecz przez czynniki zewnętrzne: mitygację w oprogramowaniu, brak dostępnej podatnej konfiguracji albo koszt obliczeniowy przekraczający możliwości pojedynczej stacji. We wszystkich trzech przypadkach RFC 9580 albo usuwa przyczynę podatności (SHA-1 w MDC, niedookreślenie ElGamala), albo ogranicza jej powierzchnię (blokada degradacji dla kluczy v6), co dodatkowo tłumaczy, dlaczego odtworzenie ataku wobec współczesnego, zgodnego ze standardem oprogramowania jest utrudnione.

Rozdział 5

Podsumowanie

Dysponując wynikami badania wydajności oraz analizy bezpieczeństwa dla obu programów, można przejść do ostatecznego porównania PGP i age.

Pod względem wydajności czasowej age osiąga niższe czasy niż PGP dla małych plików; wraz ze wzrostem rozmiaru różnica ta maleje. PGP nadrabia tę stratę w procesie deszyfrowania, w którym dla większych plików wykonuje operacje szybciej niż age. Mimo to w większości badanych przypadków age okazał się szybszą opcją. Należy zaznaczyć, że oba programy działały wyłącznie z opcjami domyślnymi. Użycie dodatkowych parametrów konfiguracyjnych mogłoby wpłynąć na wyniki porównania; uwaga ta dotyczy przede wszystkim PGP, który udostępnia znacznie więcej opcji.

Gdy jednak spojrzeć na całość procesu szyfrowania i deszyfrowania, bez pomijania komunikacji z kluczem sprzętowym, różnice między oboma rozwiązaniami zacierają się dla małych plików. Narzut czasowy komunikacji między aplikacją kliencką a bezpiecznym podzespołem klucza wynosi średnio około 78 ms. Dla plików od 100 B do 64 KiB najdłuższy czas operacji to 9,18 ms przy szyfrowaniu i 5,64 ms przy deszyfrowaniu — odpowiednio zaledwie ok. 12% i 7% narzutu komunikacji. Oznacza to, że dla małych plików o czasie end-to-end decyduje komunikacja ze sprzętem, a różnica algorytmiczna między PGP a age pozostaje praktycznie nieistotna. Rozbieżność ujawnia się dopiero dla większych plików, gdy czasy operacji rosną: przy szyfrowaniu lepiej wypada age, przy deszyfrowaniu PGP — jednak różnica ta jest niewielka.

Przechodząc do podsumowania analizy bezpieczeństwa: niemal wszystkie z ośmiu przeanalizowanych klas podatności GPG mają jedno wspólne źródło: brak domyślnego uwierzytelnienia (AEAD) oraz twardą kompatybilność wsteczną PGP. Tryb CFB bez ochrony integralności prowadzi do podatności takich jak *malleability* czy ataki typu *oracle*. Współdzielony sekret i brak kryptograficznego wiązania pól umożliwiają atak KO. Stare prymitywy (3DES, SHA-1 w S2K) pozostają w standardzie, ponieważ nie można ich usunąć bez utraty kompatybilności wstecznej. Jawne metadane, kompresja przed szyfrowaniem, rozrośnięty format i pełna elastyczność to konsekwencje tego samego podejścia: dodawaj użytkownikowi dodatkowe możliwości bez usuwania nawet skompromitowanych rozwiazań kryptograficznych.

Ze względu na lata swojej służby i błędne decyzje projektowe podjęte u samych podstaw projektu PGP ponosi dziś tego konsekwencje. Zmiany standardu wprowadzane w odpowiedzi na wykrywane podatności nie są dostatecznie szybkie, a co istotniejsze, nie odnoszą się do sedna problemu, lecz jedynie obniżają powagę zagrożenia z krytycznej do umiarkowanej. Dobrze ilustruje to RFC 9580: standard wprowadza mechanizmy naprawcze (SEIPD v2 z AEAD, Argon2 jako S2K, PKESK v6 ukrywający *Key ID*, wykrywanie kolizji SHA-1), lecz zachowuje kompatybilność z RFC 4880, w tym SEIPD v1, tryb CFB i 3DES. W efekcie wszystkie mitygacje pozostają opcjonalne, a możliwość niepoprawnego użycia realna.

age podąża drogą odwrotną: jedno AEAD (ChaCha20-Poly1305), jeden memory-hard KDF, jedna krzywa, jeden format i zero możliwości wprowadznia zmian w tym aspekcie przez użytkownika. Dzięki temu większość omawianych podatności jest strukturalnie niewyrażalna, dlatego, że nigdy nie powstał mechanizm, który by je umożliwiał. W tym porównaniu age jawi się jako narzędzie zaprojektowane z myślą o najwyższych standardach bezpieczeństwa, realizujące swoje jedyne zadanie, szyfrowanie plików, w sposób bardzo dobry.

age wypada zatem co najmniej równie dobrze jak PGP pod względem wydajności, a pod względem bezpieczeństwa zdecydowanie go deklasuje. Daje to mocne argumenty za tezą, że w 2026 r. rozwiązania umożliwiające szyfrowanie informacji powinny zrezygnować z PGP i wdrożyć age jako główne narzędzie szyfrujące.

Należy zaznaczyć, że praca przyjmuje podejście czysto akademickie do użycia obu programów. W przyszłości warto przeprowadzić badania nad wdrażaniem age w rzeczywistych systemach, w których obecnie używany jest PGP, w szczególności nad kosztem takiej migracji, gotowością na kryptografię postkwantową oraz możliwością formalnej weryfikacji obu implementacji.

Bibliografia

[1] „age Encryption Format", Community Cryptography Specification Project (C2SP), term. wiz. 10 lip. 2026. adr.: https://c2sp.org/age.

[2] Bellare, M. i Namprempre, C., *Authenticated Encryption: Relations among notions and analysis of the generic composition paradigm*, Cryptology ePrint Archive, Paper 2000/025, 2000. adr.: https://eprint.iacr.org/2000/025.

[3] Bhargavan, K. i Leurent, G., *On the Practical (In-)Security of 64-bit Block Ciphers: Collision Attacks on HTTP over TLS and OpenVPN*, Cryptology ePrint Archive, Paper 2016/798, 2016. DOI: 10.1145/2976749.2978423. adr.: https://eprint.iacr.org/2016/798.

[4] Bruseghini, L., Huigens, D. i Paterson, K. G., „Victory by KO: Attacking OpenPGP Using Key Overwriting", w *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '22, Los Angeles, CA, USA: Association for Computing Machinery, 2022, s. 411–423, ISBN: 9781450394505. DOI: 10.1145/3548606.3559363. adr.: https://doi.org/10.1145/3548606.3559363.

[5] Cloud Security Alliance AI Safety Initiative, „Vibe Coding Security Crisis: Credential Sprawl and SDLC Debt", Cloud Security Alliance AI Safety Initiative, 2026. term. wiz. 24 czer. 2026. adr.: https://labs.cloudsecurityalliance.org/wp-content/uploads/2026/03/CSA_research_note_AI_generated_code_security_vibe_coding_20260331-csa-styled.pdf.

[6] CSAI Foundation i Cloud Security Alliance AI Safety Initiative, „Vibe Coding's Security Debt: The AI-Generated CVE Surge", Cloud Security Alliance AI Safety Initiative, 2026. term. wiz. 24 czer. 2026. adr.: https://labs.cloudsecurityalliance.org/wp-content/uploads/2026/04/CSA_research_note_ai-generated-code-vulnerability-surge_20260404-csa-styled.pdf.

[7] De Feo, L., Poettering, B. i Sorniotti, A., „On the (In)Security of ElGamal in OpenPGP", w *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '21, Virtual Event, Republic of Korea: Association for Computing Machinery, 2021, s. 2066–2080, ISBN: 9781450384544. DOI: 10.1145/3460120.3485257. adr.: https://doi.org/10.1145/3460120.3485257.

[8] Dworkin, M. J., „Recommendation for Block Cipher Modes of Operation: Methods and Techniques", NIST, NIST Special Publication 800-38A, 2001. DOI: 10.6028/NIST.SP.800-38A.

[9] European Union Agency for Cybersecurity (ENISA), „ENISA Threat Landscape 2025", ENISA, 2025. term. wiz. 24 czer. 2026. adr.: https://www.libertify.com/interactive-library/enisa-threat-landscape-2025-cybersecurity-analysis/.

[10] Finney, H., Donnerhacke, L., Callas, J., Thayer, R. L. i Shaw, D., *OpenPGP Message Format*, RFC 4880, list. 2007. DOI: 10.17487/RFC4880. adr.: https://www.rfc-editor.org/info/rfc4880.

[11] Grigg, J. „age-plugin-yubikey: YubiKey Plugin for age", term. wiz. 7 lip. 2026. adr.: https://github.com/str4d/age-plugin-yubikey.

[12] Halpin, H., „SoK: why Johnny can't fix PGP standardization", w *Proceedings of the 15th International Conference on Availability, Reliability and Security*, ser. ARES 2020, ACM, sierp. 2020, s. 1–6. DOI: 10.1145/3407023.3407083. adr.: http://dx.doi.org/10.1145/3407023.3407083.

[13] Housley, R. i Schaad, J., *Advanced Encryption Standard (AES) Key Wrap Algorithm*, RFC 3394, paź. 2002. DOI: 10.17487/RFC3394. adr.: https://www.rfc-editor.org/info/rfc3394.

[14] IBM Security i Ponemon Institute, „Cost of a Data Breach Report 2025", IBM, 2025. term. wiz. 24 czer. 2026. adr.: https://www-api.ibm.com/adobe/assets/urn:aaid:aem:607b9590-38e0-4c91-b433-aa8a17f5b5e8/original/as/cost-of-a-data-breach-2025-full-report.pdf.

[15] IBM X-Force. „2026 X-Force Threat Intelligence Index: Making the case for securing identities, AI-enhanced detection and proactive risk management", term. wiz. 15 sierp. 2026. adr.: https://www.ibm.com/think/x-force/threat-intelligence-index-2026-securing-identities-ai-detection-risk-management.

[16] Kelsey, J., „Compression and Information Leakage of Plaintext", w *Fast Software Encryption*, Daemen, J. i Rijmen, V., red., Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, s. 263–276, ISBN: 978-3-540-45661-2.

[17] Leurent, G. i Peyrin, T., „SHA-1 is a shambles: first chosen-prefix collision on SHA-1 and application to the PGP web of trust", w *Proceedings of the 29th USENIX Conference on Security Symposium*, ser. SEC'20, USA: USENIX Association, 2020, ISBN: 978-1-939133-17-5.

[18] Linux man-pages project, *clock_gettime(2) — Linux Programmer's Manual*, Linux man-pages 6.18, 2026. term. wiz. 7 lip. 2026. adr.: https://man.archlinux.org/man/clock_gettime.2.en.

[19] Lloyd, J. i Botan Contributors, *Botan: Crypto and TLS for Modern C++*, wer. 3.13.0, 2024. term. wiz. 27 sierp. 2026. adr.: https://botan.randombit.net/.

[20] Mbed TLS Contributors. „Mbed TLS: An Open-Source, Portable TLS and Cryptography Library", TrustedFirmware.org, term. wiz. 7 lip. 2026. adr.: https://github.com/Mbed-TLS/mbedtls.

[21] Microchip Technology Inc., *ATECC508A CryptoAuthentication Device Complete Data Sheet*, Dokument nr DS20005927A, Microchip Technology Inc., 2017. term. wiz. 7 lip. 2026. adr.: https://www.microchip.com/en-us/product/ATECC508A.

[22] Microchip Technology Inc. „CryptoAuthLib: Microchip CryptoAuthentication Library", Microchip Technology Inc., term. wiz. 7 lip. 2026. adr.: https://github.com/MicrochipTech/cryptoauthlib.

[23] Mister, S. i Zuccherato, R., *An Attack on CFB Mode Encryption As Used By OpenPGP*, Cryptology ePrint Archive, Paper 2005/033, 2005. adr.: https://eprint.iacr.org/2005/033.

[24] Nikitin, K., Barman, L., Lueks, W., Underwood, M., Hubaux, J.-P. i Ford, B., *Reducing Metadata Leakage from Encrypted Files and Communication with PURBs*, 2019. arXiv: 1806.03160 [cs.CR]. adr.: https://arxiv.org/abs/1806.03160.

[25] NXP Semiconductors, *AN12413: SE050 APDU Specification*, NXP Semiconductors, 2021. term. wiz. 7 lip. 2026. adr.: https://www.nxp.com/docs/en/application-note/AN12413.pdf.

[26] Percival, C., *Stronger Key Derivation via Sequential Memory-Hard Functions*, Presented at BSDCan 2009, 2009. adr.: https://www.tarsnap.com/scrypt/scrypt.pdf.

[27] Poddebniak, D. i in., „Efail: Breaking S/MIME and OpenPGP Email Encryption using Exfiltration Channels", w *27th USENIX Security Symposium (USENIX Security 18)*, Baltimore, MD: USENIX Association, sierp. 2018, s. 549–566, ISBN: 978-1-939133-04-5. adr.: https://www.usenix.org/conference/usenixsecurity18/presentation/poddebniak.

[28] Ruoti, S., Andersen, J., Zappala, D. i Seamons, K., *Why Johnny Still, Still Can't Encrypt: Evaluating the Usability of a Modern PGP Client*, 2016. arXiv: 1510.08555 [cs.CR]. adr.: https://arxiv.org/abs/1510.08555.

[29] Sassaman, L., Patterson, M. L., Bratus, S. i Locasto, M. E., „Security Applications of Formal Language Theory", *IEEE Systems Journal*, t. 7, nr. 3, s. 489–500, 2013. DOI: 10.1109/JSYST.2012.2222000.

[30] SOPHOS, „THE STATE OF RANSOMWARE 2025", SOPHOS, 2025. term. wiz. 24 czer. 2026. adr.: https://assets.sophos.com/X24WTUEQ/at/9brgj5n44hqvgsp5f5bqcps/sophos-state-of-ransomware-2025.pdf.

[31] SOS Ransomware. „Double extortion: understanding and protecting yourself against this ransomware threat", term. wiz. 24 czer. 2026. adr.: https://sosransomware.com/en/cybersecurity/double-extortion-understanding-and-protecting-yourself-against-this-ransomware-threat/.

[32] Steube, J. i Gristina, G., *hashcat: World's Fastest and Most Advanced Password Recovery Utility*, https://hashcat.net/hashcat/, Wersja 7.1.2, wydana 23 sierpnia 2025, 2025. adr.: https://github.com/hashcat/hashcat/releases/tag/v7.1.2.

[33] Thach, H. i TinyUSB Contributors. „TinyUSB: An Open-Source Cross-Platform USB Stack for Embedded Systems", term. wiz. 7 lip. 2026. adr.: https://github.com/hathach/tinyusb.

[34] The GnuPG Project, *The GNU Privacy Guard (GnuPG)*, 2026. term. wiz. 3 lip. 2026. adr.: https://www.gnupg.org.

[35] The Go Authors. „Package crypto/ecdh", The Go Programming Language, term. wiz. 20 lip. 2026. adr.: https://pkg.go.dev/crypto/ecdh.

[36] Unit 42 (Palo Alto), „Global Incident Response Report 2026", Palo Alto, 2026. term. wiz. 24 czer. 2026. adr.: https://www.paloaltonetworks.com/content/dam/pan/en_US/assets/pdf/unit42/Unit42-Global-Incident-Response-Report.pdf.

[37] Valsorda, F. „age and Authenticated Encryption", term. wiz. 24 czer. 2026. adr.: https://words.filippo.io/age-authentication/.

[38] Valsorda, F. i Cox, B., *age: A Simple, Modern and Secure File Encryption Tool*, wer. 1.3.1, 2021. term. wiz. 3 lip. 2026. adr.: https://github.com/FiloSottile/age.

[39] Valsorda, F. i Cox, B. „age Encryption Format, Version 1", C2SP (Community Cryptography Specification Project), term. wiz. 4 lip. 2026. adr.: https://https://github.com/C2SP/C2SP/blob/main/age.md.

[40] Veracode, „2025 GenAI Code Security Report", Veracode, 2025. term. wiz. 24 czer. 2026. adr.: https://www.veracode.com/wp-content/uploads/October-2025-GenAI-Code-Security-Report-Update.pdf.

[41] Verizon Business, „2026 Data Breach Investigations Report", Verizon, Raport branżowy, 2026. term. wiz. 24 czer. 2026. adr.: https://www.verizon.com/business/resources/Tfc1/reports/2026-dbir-data-breach-investigations-report.pdf.

[42] Wouters, P., Huigens, D., Winter, J. i Yutaka, N., *OpenPGP*, RFC 9580, lip. 2024. DOI: 10.17487/RFC9580. adr.: https://www.rfc-editor.org/info/rfc9580.

[43] Zimmermann, P., Atkins, D. i Stallings, W., *PGP Message Exchange Formats*, RFC 1991, sierp. 1996. DOI: 10.17487/RFC1991. adr.: https://www.rfc-editor.org/info/rfc1991.

Spis rysunków

1 Schemat sprzętowego klucza kryptograficznego 19
2 Wysokopoziomowy diagram architektury całego środowiska użytego do porównania programów 22
3 Porównianie czasu wykonania operacji ECDH systemowo oraz sprzętowo dla GPG i age na komputerze stacjonarnym i laptopie 28
4 Zestawienie wykresów czasu szyfrowania plików za pomocą GPG i age na komputerze stacjonarnym i laptopie 30
5 Zestawienie wykresów czasu deszyfrowania plików za pomocą GPG i age na komputerze stacjonarnym i laptopie 31
6 Porównanie wielkości szyfrogramów GPG i age względem wagi szyfrowanych plików . 32

Spis tabel